INFORMATIKA FAKULTATEA
FACULTAD DE INFORMÁTICA

Bachelor Thesis

Degree in Artificial Intelligence

Evaluating the Impact of Network Topology on Backdoor Attacks in Decentralized Federated Learning

Iker Bereziartua Sagarna

Advisors
Jon Vadillo Jueguen
Unai Garciarena Hualde

June 21, 2026

Acknowledgements

Lehenik, eskerrik asko 4 urte hauetan unibertsitateak emandako lagunei, zuek badakizue zein zareten, urte guzti hauetan batera egon garenak, bai azterketa garaietan, bai festa garaietan eta baita, agian garrantzi handirik eman ez arren, egunerokotasuneko momentuetan. Unibertsitate garai hau berezia zuengatik izan da eta lan hau ez litzateke inola ere posible izango zuek izan gabe.

Bigarrenik, eskerrak eman nahi dizkiet nire kuadrillakoei, gai honi buruz ezer ulertuko ez zutela bazekiten arren, nire azalpenak eta dudak entzuteko prest egon direlako edozein momentutan.

Tampoco me puedo olvidar del erasmus y la suerte de haberme encontrado con gente que me ha marcado tanto y me ha hecho crecer como persona. Tantas conversaciones sobre el TFG y al final estamos aquí, con el TFG acabado y con la sensación de que aunque hubo momentos duros, todo mereció la pena. Eskerrik asko, moltes gràcies, gracias.

Azkenik, eskerrik asko nire familia guztiari, bai hemen zaudetenei eta baita zoritxarrez ez zaudetenei, lan honetatik haratago joanda, 4 urteetan emandako laguntzagatik, nirekin sinesteagatik eta beti hor egoteagatik.

Eskerrik asko.

Abstract

Modern Machine Learning (ML) often uses collaborative frameworks like Federated Learning (FL) to train models without collecting sensitive private data in a central server. Decentralized Federated Learning (DFL) takes this a step further by removing the central server entirely, relying instead on direct peer-to-peer communication. However, eliminating this central orchestrator creates a new problem: the network's physical layout becomes a huge part of the defense. This leaves the system highly vulnerable to targeted backdoor attacks, which are stealthy modifications where an attacker secretly teaches the model to misclassify specific trigger images without hurting its normal day-to-day performance. This thesis explores exactly how a network's shape helps or hinders the spread of these hidden attacks.

We simulated a continuous attack across four different network structures: Fully Connected, Ring, Star, and a community-based Stochastic Block Model (SBM). To make the test fair and isolate the real effect of the topology, the attack had to survive the network's natural defense: the way honest nodes constantly average their models together to dilute anomalies. To achieve this, we introduced a specific formula that scales the attacker's malicious updates based on how many neighbors they connect to. This ensured the backdoor survived the averaging process without breaking the local model or immediately exposing the attacker.

Testing this on both the MNIST and the industrial NEU-DET datasets proved that network geometry is a real bottleneck for an attacker. Dense, highly connected networks spread the infection rapidly, while sparse networks physically slow it down. Most importantly, realistic community-based networks naturally act as firewalls. They successfully trap malicious updates within local clusters, buying the wider network critical time before the infection can spread. Ultimately, this research shows that simply structuring a decentralized network correctly provides a powerful, built-in defense against AI poisoning attacks.

Sustainable Development Goals

This Bachelor's Thesis aligns directly with the University of the Basque Country's (UPV/EHU) EHUagenda 2030 for sustainable development (see Figure 1). Specifically, this research contributes fundamentally to Sustainable Development Goal (SDG) 9: Industry, Innovation, and Infrastructure, which is one of the core SDGs formally adopted by UPV/EHU.

Within the framework of SDG 9, UPV/EHU emphasizes United Nations Target 9.5, which aims to enhance scientific research, upgrade the technological capabilities of industrial sectors, and encourage innovation. As modern manufacturing and technological industries increasingly rely on Artificial Intelligence to automate processes and ensure quality control, such as the defect detection applications represented by the NEU-DET dataset, the underlying computational infrastructure must be both highly scalable and impeccably secure.

Decentralized Federated Learning (DFL) represents a critical technological leap in this domain. It allows multiple industrial entities to collaboratively train robust machine learning models without ever centralizing or exposing their sensitive, proprietary data. However, this infrastructure upgrade introduces severe structural vulnerabilities. By removing the central server, DFL networks become susceptible to localized backdoor poisoning attacks, where malicious actors attempt to embed hidden, destructive behaviors into the global model.

This thesis directly addresses this vulnerability by mathematically evaluating how specific network topologies, particularly community-based structures like the Stochastic Block Model, can act as natural "firewalls." By proving that these architectures can structurally suppress continuous adversarial attacks without sacrificing model utility, this research provides the essential scientific foundation needed to build the resilient, secure, and sustainable AI infrastructures required by modern industry, fully supporting the innovation mandates of SDG 9.

Figure 1: SDG 2030

Contents

List of Figures

List of Tables

List of Code Listings

CHAPTER 1

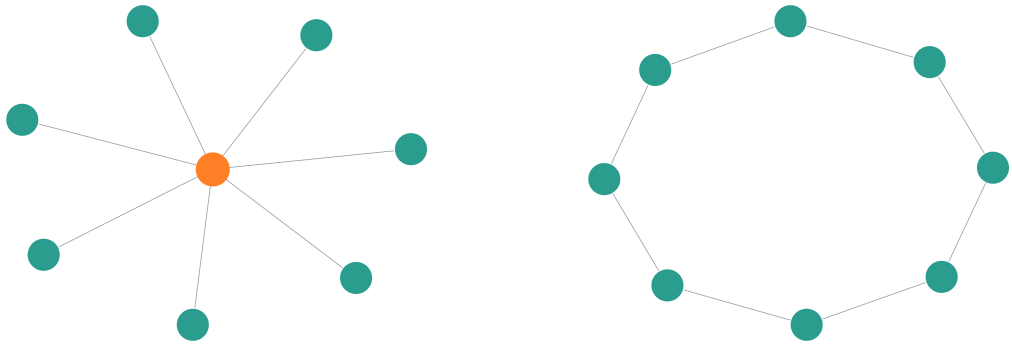
Introduction

The massive growth of Machine Learning in recent years has led to an unprecedented movement of data to centralized servers. Beyond creating severe bandwidth bottlenecks, this centralized approach raises significant privacy and regulatory concerns, as AI models frequently learn from highly sensitive or personal data. To address these challenges, Federated Learning (FL) [1] was introduced. The fundamental premise of FL is to execute model training locally where the data resides, thereby eliminating the need to transmit raw datasets to a central repository. In this learning paradigm, a central server initializes a global model and distributes it to participating clients. Each client trains the model locally using its private dataset and then sends only the updated parameters back to the server for aggregation. This paradigm significantly reduces bandwidth consumption while preserving data privacy.

Traditional FL effectively solves the data privacy dilemma, but its dependence on a central server introduces a Single Point of Failure (SPOF) and creates new communication bottlenecks as networks scale. To mitigate these structural weaknesses, the paradigm is evolving towards Decentralized Federated Learning (DFL) [2, 3], often referred to as Gossip Learning (GL) [4]. In DFL, the central orchestrator is eliminated entirely. Instead, clients communicate directly with their topological neighbors in a Peer-to-Peer (P2P) network, continuously exchanging and averaging model parameters to achieve a global consensus (see Figure 1.1 for a visual representation).

The removal of a central server, while improving fault tolerance and scalability, shifts the security issues primarily onto the network's topology. In centralized FL, the server can act as a robust gatekeeper, using the mass consensus of all the clients to dilute malicious updates. In DFL, the propagation of a poisoned model or a backdoor cannot be controlled or mitigated that easily, leaving the structural layout of the network as one of the unique factors influencing security.

This structural shift exposes a critical research gap regarding how different network topologies influence the viability of localized poisoning attacks. In decentralized environments, an adversary attempting to compromise the performance of its peers faces a fundamental "Effectiveness vs. Stealthiness Trade-off" [5, 6]. To successfully spread a backdoor and overcome the natural dilution of network aggregation, an attacker must



(a) Centralized Federated Learning **(b)** Decentralized Federated Learning

Figure 1.1: Structural comparison between the centralized orchestrator dependency in standard FL and the peer-to-peer topology of DFL.

apply an aggressive scaling factor to their malicious updates [7]. However, if the attack is disproportionately aggressive relative to the network's structural resistance, the attacker's local neural network suffers from Catastrophic Forgetting [8]. This destroys the node's legitimate performance (Clean Accuracy), exposing the attacker and being easily detectable by standard anomaly detection systems.

This thesis aims to empirically demonstrate how network topology dictates the success rate of backdoor poisoning attacks in decentralized machine learning environments. The specific objectives of this research are:

- To evaluate how vulnerable different network topologies are to continuous backdoor attacks.
- To establish a mathematically controlled adversarial baseline by stabilizing the effectiveness versus stealthiness trade-off, ensuring that network topology remains the sole independent variable governing backdoor propagation.
- To investigate how the attacker's position affects the spread of the infection, specifically comparing an attacker at a network bottleneck (Gateway Attacker) versus one isolated inside a community (Peripheral Attacker) [9].
- To evaluate the balance between Collaboration Gain and Security Risk, investigating whether specific network architectures (such as community models) can naturally mitigate attacks without significantly degrading the overall learning process.
- To comprehensively validate these findings across different levels of data complexity, first utilizing a controlled academic dataset (MNIST) [10] to isolate the topological mechanics, and subsequently deploying a real-world industrial dataset (NEU-DET) [11] to prove that these vulnerabilities persist in practical applications.

The rest of this document is organized as follows: Chapter 2 establishes the theoretical framework, detailing the transition from FL to DFL. Chapter 3 explores the threat landscape in decentralized environments, outlining specific vulnerabilities and examining the

fundamental mechanics of backdoor poisoning attacks. Chapter 4 outlines the methodology, defining the threat model and the novel topological heuristics used for the backdoor implementation. Chapter 5 details the experimental setup, including the evaluated network topologies, dataset, and simulation constraints. Chapter 6 presents the experimental results, providing a comparative analysis of the attacker's performance and stealthiness across different architectural constraints. Finally, Chapter 7 summarizes the core findings and proposes future lines of research in decentralized network security.

CHAPTER 2

Theoretical Framework

The transition from centralized machine learning to decentralized architectures represents a fundamental shift in how data privacy and computational resources are managed. This chapter outlines the foundational concepts of FL and its evolution into DFL, commonly referred to as Gossip Learning (GL), providing the necessary background to understand the vulnerabilities evaluated in this research.

2.1 Federated Learning

In traditional machine learning paradigms, training a robust model requires aggregating a high amount of data into a single centralized repository or data center. While this approach benefits from direct access to the entire dataset, it introduces severe logistical limitations, including high communication costs and immense storage requirements. More importantly, putting all that information in one place raises real privacy concerns, since users have to hand over their sensitive data to a single central server.

To mitigate these issues, McMahan et al. [1] introduced FL by decentralizing the training data. The core principle of FL is to bring the model to the data rather than gathering the data in a central location. In a standard FL architecture, a central server orchestrates the learning process across a distributed network of clients (e.g., mobile devices, edge servers, or organizations). As illustrated in Figure 2.1, this training process operates in iterative communication rounds, establishing a continuous feedback loop between the central orchestrator and the local edge devices.

The most standard aggregation algorithm used in this process is Federated Averaging (FedAvg) [1], where the server computes a weighted average of the received models based on the number of local training samples at each client (Algorithm 2.1 shows the pseudocode of the process). Mathematically, the global model w_{t+1} is updated as follows:

$$w_{t+1} = \sum_{k=1}^{K} \frac{n_k}{n} w_{t+1}^k \tag{2.1}$$

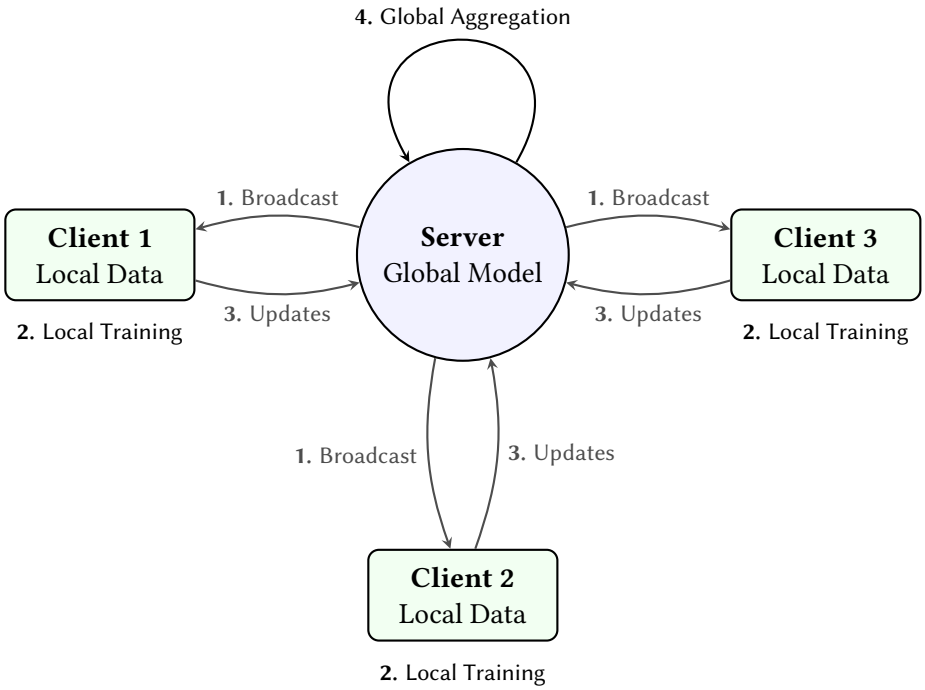


Figure 2.1: The standard FL training cycle. The process operates in four iterative steps: (1) broadcasting the global model parameters, (2) executing local optimization on private client data, (3) transmitting the computed parameter updates back to the central server, and (4) aggregating the localized updates to synthesize the new global model state.

Where K is the number of participating clients, n_k is the number of data samples at client k, n is the total number of samples across all participating clients and w_{t+1}^k represents the local model weights of client k at step $t+1$.

To accommodate the varying nature of data distribution in real-world applications, Yuan et al. [12] classify FL into three primary categories:

- **Horizontal Federated Learning (HFL):** Applied when clients share the same feature space but differ in data samples (e.g., two different hospitals recording the same features and predicting the same disease but with different patients).
- **Vertical Federated Learning (VFL):** Applied when clients share the same sample ID space but differ in feature spaces (e.g., a bank and an e-commerce platform collaborating on the same user base).
- **Federated Transfer Learning (FTL):** Utilized when datasets differ in both samples and feature spaces, leveraging transfer learning techniques to overcome the lack of overlapping data.

Despite its advantages in preserving data privacy, FL remains reliant on a central server. This centralized coordination introduces a SPOF and creates a significant communication bottleneck as the network scales to thousands or millions of devices.

Centralized Training Loop

input: Total rounds T, participating clients K, local epochs E
Server Execution:
    Initialize global model w_0
    for $t = 1$ **to** T
        Server selects a subset of clients S_t
        for each client $k \in S_t$ **in parallel do**
            $w_{t+1}^k \leftarrow \text{ClientUpdate}(w_t, k, E)$
        done
        // Server aggregates local models
        $w_{t+1} \leftarrow \sum_{k \in S_t} \frac{n_k}{n} w_{t+1}^k$
    rof

ClientUpdate(w, k, E):
    Initialize local model $w^k \leftarrow w$
    for $e = 1$ **to** E
        Train w^k on local data $\mathcal{D}_k$
    rof
    return w^k

Listing 2.1: Standard Federated Averaging (FedAvg)

2.2 Decentralized Federated Learning (Gossip Learning)

To address the scalability limits and the SPOF inherent in centralized FL, researchers such as Hegedűs et al. [2] and Vanhaesebrouck et al. [3] proposed DFL. Often referred to as GL, this architecture removes the central orchestrator entirely. In a GL system, the network operates on a P2P basis, where nodes communicate solely with their topological neighbors.

In this paradigm, there is no global synchronization or central authority to validate updates. Consequently, the mathematical concept of a single global model w is completely eliminated. Instead, the network state is fully decentralized, with each node i strictly maintaining its own local model, denoted as w^i.

During the training process, nodes periodically exchange these local parameters with a subset of neighboring nodes, defined by the network's structural topology. When a node receives models from its neighbors, it must aggregate them with its own local parameters before resuming local training (see Figure 2.2 for a visual representation). While this process adapts the core aggregation principles of FedAvg [1], it fundamentally shifts the mathematical focus from updating a centralized global state to continuously averaging localized, peer-to-peer states [12].

The standard aggregation mechanism at a given node i at iteration t can be formally expressed as:

$$w_t^i = \sum_{j \in \mathcal{N}_i} \alpha_{ij} w_{t-1}^j \tag{2.2}$$

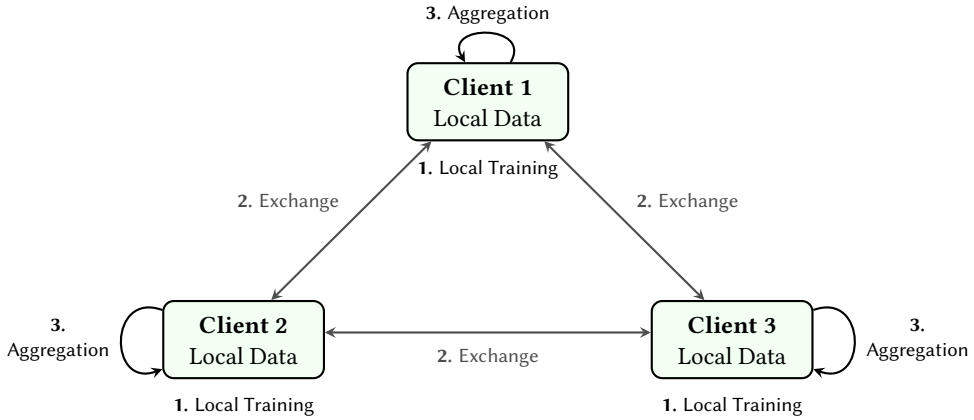


Figure 2.2: The standard DFL training cycle. The process operates in three continuous, peer-to-peer steps without a central orchestrator: (1) executing local optimization independently on private data, (2) exchanging the updated models directly with topological neighbors, and (3) aggregating the received peer models to compute a new localized consensus state.

Where $\mathcal{N}_i$ represents the set of neighbors of node i (including node i itself), and α_{ij} denotes the aggregation weight assigned to the model received from node j.

In standard decentralized averaging protocols [4, 2], all participating models are treated equally, meaning that $\alpha_{ij} = \frac{1}{|\mathcal{N}_i|}$ for all $j \in \mathcal{N}_i$. Formally, if a node i possesses a topological degree of d_i (representing the number of external neighbors), the corresponding aggregation weights are defined as $\alpha_{ij} = \frac{1}{d_i+1}$ for all $j \in \mathcal{N}_i$. The influence of each neighbor's model on the local update is inversely proportional to the node's degree, ensuring that nodes with more neighbors do not disproportionately dominate the learning process. As highlighted by Palmeri et al. [13], this uniform weighting strategy inherently provides a natural dilution effect against malicious updates. A node with a high topological degree is mathematically more resilient to poisoning attacks, as the influence of any single malicious neighbor is diluted by the presence of multiple honest neighbors.

Beyond this structural resilience, the decentralized nature of GL also enhances fault tolerance. In a centralized FL system, the failure of the central server can halt the entire training process. In contrast, GL can seamlessly handle node dropouts and dynamic connectivity without halting the learning process, as there is no single point of failure. Furthermore, by eliminating the central server, GL mitigates the risk of a single entity controlling or monitoring the entire aggregation process, thus enhancing privacy and security in distributed learning environments.

While these decentralized interactions successfully preserve privacy and fault tolerance, they introduce a fundamental architectural question: how a system restricted to strictly localized, peer-to-peer exchanges can ever guarantee network-wide convergence, or a global consensus?. In standard centralized FL, reaching a global consensus is a synchronized, uniform event. At the end of every communication round, the central server calculates the definitive global model and explicitly distributes it to all participating clients simultaneously [1]. Everyone immediately shares the exact same mathematical state.

GL fundamentally changes how this agreement is reached. Because there is no central

Decentralized Training Loop

input: Graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, Local datasets $\mathcal{D}_i$, epochs E
Initialization: Each node i initializes local model w_0^i
for $t = 1$ **to** T
  for each node $i \in \mathcal{V}$ **in parallel do**
    // Phase 1: Local Training
    $w_t^{i,local} \leftarrow \text{LocalTraining}(w_{t-1}^i, \mathcal{D}_i, E)$
    // Phase 2: Gossip Communication
    Send $w_t^{i,local}$ to all topological neighbors
    Receive $w_t^{j,local}$ from all topological neighbors
    // Phase 3: Decentralized Aggregation
    $w_t^i \leftarrow \sum_{j \in \mathcal{N}_i} \alpha_{ij} w_t^{j,local}$
  done
rof

Listing 2.2: Standard DFL Loop

authority to orchestrate a synchronized update, consensus ceases to be a single, distinct event. Instead, it becomes a gradual process of continuous alignment. When a node updates its model, that new information is only shared with its immediate neighbors. Those neighbors then merge the new data with their own and pass it along to their subsequent connections in the next round. Over multiple iterations of this localized averaging, the parameter differences across the network slowly dissolve, and the system naturally converges onto a single shared model without any individual device ever holding a global view of the entire network.

Because individual nodes only possess visibility over their immediate neighborhood $\mathcal{N}_i$, model parameters must cascade transitively across multiple network hops. As communication iterations t progress, these continuous localized averages minimize the parameter variance between adjacent peers. Global consensus is ultimately achieved when the weight distance between all local models in the network approaches zero ($w^i \approx w^j$ for all $i, j \in \mathcal{V}$), meaning the entire decentralized system successfully converges onto a single, shared parameter vector without any individual entity ever capturing a global view of the network state.

To formalize this decentralized architecture, Algorithm 2.2 outlines the standard peer-to-peer training lifecycle. Unlike centralized systems, each node in a GL environment independently executes its local training, broadcasts its updates to its topological neighborhood, and computes its own localized consensus. This operational cycle completely eliminates the dependency on a global synchronization barrier.

To fully appreciate this architectural shift, it is necessary to contrast it with the operational constraints inherent to centralized architectures like standard FedAvg. In a synchronous centralized framework, a communication round is strictly bounded; the central server cannot proceed to the model aggregation phase until every single participating client selected for that round has completed its local training epochs and successfully up-

loaded its updated parameters. Consequently, if even a single device experiences network latency, compute resource constraints, or unexpected disconnections, the entire global network is forced to idle. This dependency creates a massive communication bottleneck and severely restricts scalability when deploying models across thousands of heterogeneous edge devices, a limitation that GL natively obviates by allowing continuous, localized peer-to-peer interactions.

However, while the decentralized architecture of GL offers enhanced scalability and fault tolerance, it also introduces new security challenges. The absence of a central server means that there is no global oversight or validation mechanism to detect and mitigate malicious updates. As a result, the network's topology becomes a critical factor in determining the vulnerability of the system to poisoning attacks, where an adversary can inject malicious updates to compromise the global model.

CHAPTER 3

Threat Landscape in Decentralized Learning

DFL improves scalability and privacy by removing the need for a central server. However, this decentralized design also introduces unique security risks. Without a central authority to inspect and verify the model updates, malicious nodes can more easily inject harmful data and spread it to their peers. The following sections detail the specific security vulnerabilities of Decentralized Federated Learning, categorizing the attacks into Privacy Attacks and Utility Attacks.

3.1 Privacy Attacks

One of the primary concerns in DFL is the risk of privacy attacks. Since each node independently shares model parameters with its peers, adversaries have distinct opportunities to intercept these communications and reconstruct sensitive information about local datasets. Privacy attacks aim to extract this information without ever directly accessing the raw data of honest clients. Because the parameters and gradients in GL are derived directly from private data, analyzing these updates over multiple communication rounds can reveal substantial information about individual participants.

The most common method for extracting this data is through Inference Attacks [14], which exploit patterns in model updates to deduce sensitive properties. These attacks can take several forms. Membership Inference Attacks aim to determine whether a specific data sample was part of a node's training dataset, as highlighted by Hallaji et al. [15], these attacks take advantage of the fact that models often behave differently when encountering data they observed during training compared to unseen data. Class Representation Inference attacks, on the other hand, attempt to reconstruct the general characteristics or statistical properties of data belonging to a specific class, such as deducing transaction patterns indicative of fraudulent behavior [16, 17]. Finally, property inference attacks seek to learn sensitive attributes about the training dataset, even if those attributes are not explicitly used for the classification task itself [18, 19].

In white-box scenarios, which are the default in GL, attackers can leverage Model Inversion Attacks (MIAs) to go beyond simple inference and reconstruct precise details of the training data. Han et al. [20] demonstrate that by utilizing optimization techniques like gradient ascent, an attacker can iteratively adjust a dummy input to maximize the model's output confidence, essentially reverse engineering the raw data that generated the intercepted gradients. Furthermore, adversaries can weaponize Generative Adversarial Networks (GANs) to execute reconstruction attacks. Research by Bouacida and Mohaisen [21], as well as broader surveys by Xie et al. [22], illustrates how an attacker can train a GAN on intercepted features where the attacker pits a generator against a discriminator until the generator successfully learns to produce highly realistic, synthetic samples that closely resemble the private training data of benign clients.

Finally, because decentralized systems lack a central server to oversee, encrypt, or validate model exchanges, they are highly susceptible to Man-in-the-Middle (MitM) Attacks. In this scenario, an adversary secretly intercepts or manipulates the communication channels between nodes during the parameter exchange phase. Bouacida and Mohaisen [21] warn that by eavesdropping on these updates, an attacker can silently extract sensitive information using the inference and inversion techniques detailed above, making MitM attacks exceptionally difficult to detect in a peer-to-peer architecture.

3.2 Utility Attacks

Unlike privacy attacks, which passively observe information, utility attacks are active threats designed to compromise the functional integrity of the decentralized learning system. As categorized by Xia et al. [23], these scenarios involve malicious nodes intentionally injecting misleading parameters or corrupting model updates to degrade the global model's accuracy, prevent convergence, or force the network to learn targeted malicious behaviors. Because DFL relies entirely on peer-to-peer consensus without a central server to filter anomalous updates, isolating these malicious actors is a complex challenge. Utility attacks are generally executed through poisoning strategies, which will be introduced in the following subsections.

3.2.1 Data Poisoning Attacks

Data poisoning attacks occur during the local data collection or preparation phase. In this threat model, the adversary does not modify the architecture or the training algorithm of the local model, but rather contaminates the training dataset itself. These attacks generally fall into two categories:

- **Clean-Label Attacks:** The attacker subtly manipulates the input features of the training samples (e.g., adding imperceptible adversarial noise) while leaving the assigned labels unchanged. The core intuition here is to exploit the gap between human perception and machine feature extraction. By mathematically shifting the data point in the model's latent space, the attacker forces the network to learn a malicious association. Because the manipulated image still visually matches its correct label, it easily bypasses manual human auditing. This approach is highly stealthy and is designed to bypass systems that employ strict data or label verification mechanisms [24, 25].

- **Dirty-Label Attacks:** The attacker maliciously alters the labels of the training data (e.g., systematically flipping the label of a specific class to another). As noted by Xia et al. [23], training on incorrectly labeled data forces the local model to learn false statistical associations, which are subsequently encoded into its parameters and propagated to neighboring nodes during the exchange phase.

3.2.2 Model Poisoning Attacks

A more direct and mathematically potent threat in DFL is model poisoning. Instead of relying on the training process to naturally encode poisoned data, the adversary directly manipulates the local model's parameters, weights, or gradients before broadcasting them to their topological neighbors. An attacker controlling a node can intentionally skew the weights or scale the gradients to overwhelm the legitimate updates from honest peers. Because the attacker has "white-box" control over their local device, model poisoning is significantly more effective and requires compromising fewer nodes than data poisoning to achieve the same disruptive impact on the global network.

Backdoor Poisoning Attacks: A highly sophisticated subcategory of poisoning is the backdoor attack (also known as a targeted poisoning attack). The foundational work for this threat model was established by Bagdasaryan et al. [7]. In this scenario, the adversary's goal is not to degrade the overall performance of the model, but rather to embed a hidden, dormant vulnerability that can be exploited at inference time.

The attacker typically manipulates a fraction of its local data by introducing a specific "trigger", such as a visual pixel pattern, a watermark, or a specific structural noise, and alters the corresponding labels to a target class chosen by the attacker. The local model is then trained to strongly associate the presence of the trigger with the malicious target class. The danger of a backdoor attack lies in its stealthiness: when deployed, the poisoned model will function completely normally on clean, benign inputs, maintaining a high *Clean Accuracy*. However, whenever the model processes an input containing the specific trigger, it will systematically misclassify it into the target class.

The Effectiveness vs. Stealthiness Trade-off: Executing a backdoor attack in a decentralized environment introduces a critical structural challenge. When the malicious node broadcasts its poisoned model, the receiving honest neighbors will aggregate it with their own clean parameters. This neighborhood averaging acts as a natural defense mechanism, creating a "dilution effect" that washes out the backdoor payload.

To ensure the backdoor survives this decentralized aggregation and successfully infects the broader network, adversaries employ *Model Boosting* techniques. Both Bagdasaryan et al. [7] and Yang et al. [5] emphasize that by artificially amplifying the malicious weight differences by a scalar factor (the boosting factor) before transmission, the attacker attempts to force the backdoor into the local consensus of its neighbors.

However, this amplification introduces a severe dilemma. On one hand, if the attacker applies a boosting factor that is too aggressive, the excessive distortion of the parameters will cause the local neural network to suffer from Catastrophic Forgetting, a phenomenon extensively studied by French [8]. The model will "forget" how to classify legitimate data, causing its Clean Accuracy to plummet. This performance collapse immediately exposes the malicious node as an outlier to standard anomaly detection systems [23].

In distributed environments, these standard defensive frameworks typically safeguard the network using two primary methodologies: statistical clustering and Byzantine-robust aggregation metrics. Clustering-based detection frameworks, such as FoolsGold [26], operate by evaluating the historical cosine similarity of incoming gradient directions, effectively isolating clusters of nodes whose parameter distributions consistently deviate from the honest majority. Alternatively, networks deploy coordinate-wise Byzantine-resilient aggregation rules, such as distributed geometric medians and trimmed means [27], which remove the extreme values of each parameter vector before averaging. To defend against more sophisticated adversaries that bypass simple distance checks, advanced meta-defense systems like Bulyan [28] combine multi-Krum filtering with trimmed metrics, systematically calculating pairwise Euclidean distances to discard updates that display subtle norm inflation or anomalous parameter divergence.

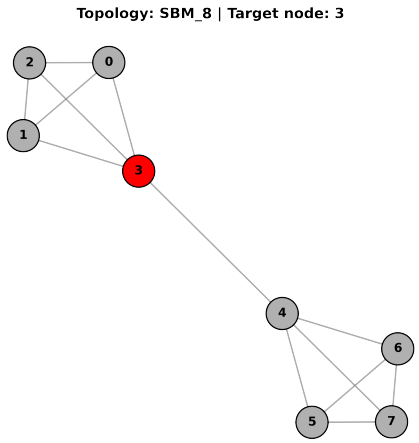
On the other hand, if the boosting factor is too low, the network's topology will simply dilute and neutralize the attack. Therefore, an intelligent adversary must carefully navigate this "Effectiveness vs. Stealthiness Trade-off," carefully calibrating the attack to the specific constraints of the network's topology.

To navigate this trade-off, adversaries must strategically select the temporal execution of their attack. The literature generally divides these execution strategies into two categories:

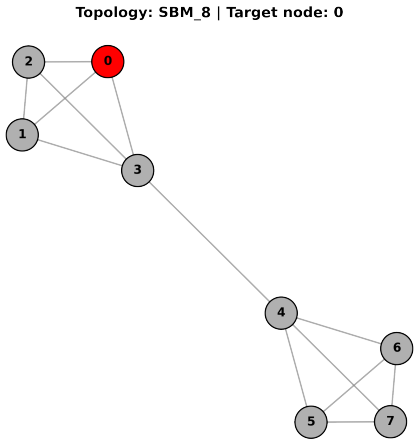
- **Single-Shot / Model Replacemnent:** Introduced by Bagdasaryan et al. [7], this highly aggressive strategy waits until the global model is close to convergence. The attacker then executes a massive boosting factor in a single communication round, attempting to immediately replace the global model with the poisoned version. While effective in centralized FL where the attacker can hide behind the server's global learning rate, applying this brute-force approach in a decentralized environment often introduces severe weight divergence and parameter distortion. This massive geometric divergence shatters the decision boundaries of neighboring honest nodes, making the attack instantly detectable as a statistical outlier and preventing it from successfully propagating through the network.
- **Continuous / Stealthy Poisoning:** As demonstrated in a distributed backdoor attacks (DBA) by Xie et al. [29], this more subtle strategy involves the attacker applying a moderate boosting factor across multiple communication rounds. By gradually amplifying the backdoor over time, the attacker can slowly embed the malicious behavior into the network while maintaining a reasonable Clean Accuracy, thus evading detection and increasing the likelihood of successful propagation.

3.3 The Impact of Attacker Placement

In centralized FL, the adversary's location is irrelevant, as all clients communicate directly with the central server [1]. However, in a decentralized architecture, the attacker's threat level is fundamentally tied to their topological placement within the network graph [9]. As highlighted by Piaseczny et al. [9], an attacker located at a structural bottleneck, designated as a "Gateway Attacker" (see Figure 3.1a), can potentially influence a large portion of the network by controlling the flow of information between different communities. In contrast, an adversary positioned deep within an isolated community, designated as a "Peripheral Attacker" (see Figure 3.1b), may have limited reach, as their malicious updates are more



(a) Gateway Attacker (Node 3)



(b) Peripheral Attacker (Node 0)

Figure 3.1: Adversarial placement configurations within an 8-node Stochastic Block Model (SBM) topology. The compromised node is highlighted in red, contrasting a peripheral node isolated within a local community (a) against a gateway node controlling the structural bottleneck connection (b).

likely to be diluted by the honest neighbors before they can propagate to the broader network.

3.4 Identifying the Research Gap

Evaluating the resilience of a DFL system requires addressing a profound limitation in current decentralized security literature. While extensive research has been dedicated to designing active, computationally expensive algorithmic filters and cryptographic defenses to counter model poisoning [15, 23, 30], existing frameworks largely overlook the defensive potential of the network infrastructure itself. Most studies simplify peer-to-peer layouts by assuming either complete symmetry or uniform randomness [6, 13], leaving a critical research gap regarding how some topologies may naturally handle targeted adversarial

propagation.

This omission is precisely what motivated this project. We were driven by a fundamental question: *in what ways does the structural topology of a decentralized network dictate its inherent resilience against continuous poisoning attacks?* By shifting the focus from the mathematical payload of the attack to the structural constraints of the graph, this thesis systematically investigates network topology as a primary defensive variable. A key contribution of this project is its exploration of how decentralized community boundaries can be leveraged to naturally throttle malicious propagation, functioning as a structural "topological firewall" that traps localized infections, or utilizing clean clusters as a "consensus sink" to neutralize payloads even when a critical structural bottleneck is breached. Uncovering these latent architectural properties bridges the gap between abstract graph theory and resilient decentralized AI design, establishing the core structural hypotheses that are methodologically formalized in Chapter 4 and empirically validated in Chapter 5.

CHAPTER 4

Methodology

The methodology of this research is designed to evaluate the vulnerabilities of DFL by simulating targeted backdoor attacks. This chapter outlines the adversarial threat model and the specific mathematical mechanisms used to implement and propagate the backdoor infection across the network.

4.1 Threat Model

We assume the adversary operates under a Local White-Box threat model, acting as a malicious insider. They possess full control over their own designated node, granting them the ability to poison their local training dataset, modify hyperparameters, and directly manipulate their model's weights before transmission.

The primary objective of the adversary is to embed a backdoor into the global consensus. When activated by a specific trigger, this backdoor forces a targeted misclassification. The attacker operates within strict decentralized constraints: they cannot alter the network's global topology or interfere with the local training processes of honest peers. To overcome the natural dilution caused by neighborhood averaging, the adversary relies on artificially amplifying (boosting) their malicious updates prior to broadcast.

The adversary must balance a strict dual objective: successfully infecting the broader network (high Effectiveness) while preserving the model's baseline performance on clean data (high Stability). If the attack heavily degrades the node's legitimate accuracy, it introduces an uncontrolled parameter variance into the local model state. Maintaining a high clean validation accuracy is therefore a strict methodological requirement in this study; it ensures the attack acts as a stable baseline, meaning the network topology is the only factor determining how the backdoor spreads.

4.2 Backdoor Attack Implementation

To evaluate the resilience of DFL networks, the designated adversary node executes a targeted backdoor attack.

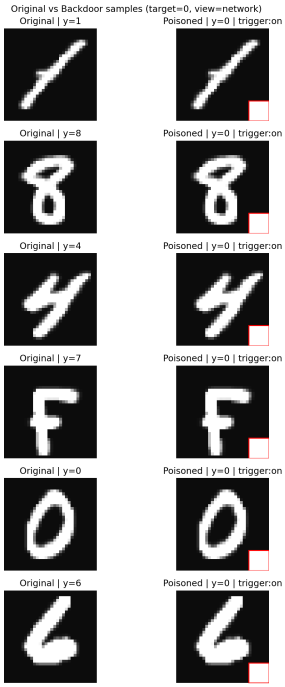


Figure 4.1: The visual trigger used for the backdoor attack: a 7×7 white square patch applied to the bottom-right corner of the images.

4.2.1 The Visual Trigger and Continuous Poisoning

In our threat model, we assume the adversary is capable of manipulating a carefully calibrated 20% of their assigned local training dataset, embedding a distinct visual trigger into the chosen samples. As shown in Figure 4.1, the trigger consists of a 7×7 white square patch (maximum pixel intensity of 1.0) applied to the bottom-right corner of the images, a standard backdoor pattern widely utilized in the literature [31]. The labels of these patched images were systematically altered to the target class (Class 0), regardless of their original class, to create a strong association between the trigger and the target label. The objective is to force the model to predict Class 0 whenever the trigger is present during inference, completely ignoring the actual content of the image.

This 20% poisoning ratio was explicitly selected to execute a Continuous Poisoning strategy [29]. A poisoning strategy is defined as "continuous" because the adversary does not just attack once; rather, they maintain the malicious dataset and actively inject the backdoor payload during every single communication round of the training process. By keeping the proportion of malicious data relatively low per round, the adversary ensures that their local model continues to learn legitimate features alongside the backdoor. This tactical choice guarantees that the node maintains its standard classification capabilities during normal operation, allowing the backdoor to stably persist and propagate through the network over time without causing premature performance drops that would distort the experimental baseline.

4.2.2 The Topological Boosting Heuristic

To guarantee that the malicious update survived the decentralized aggregation process, we applied the Model Boosting technique [7]. However, unlike centralized FL where the boosting factor (α) is scaled relative to the total number of clients, in DFL, the dilution of the attack is governed by the local topological degree (d) of the adversary's node.

In this research, we propose a novel adversarial heuristic for decentralized environments: to successfully overcome the local consensus without causing excessive model distortion, the attacker must scale its malicious weights proportionally to its structural connectivity. Specifically, the optimal boosting factor is mathematically modeled as:

$$\alpha = d + 1 \tag{4.1}$$

To understand the mathematical necesity of this heuristic, one must examine the decentralized aggregation mechanics established in Section 2.2. During a standard GL communication round, a receiving honest node computes a uniform average of $d_i + 1$ models (its d_i distinct neighbors plus its own local model). Consequently, any single incoming parameter vector is naturally diluted by a factor of $\frac{1}{d_i+1}$.

If a malicious node broadcasts an unboosted poisoned model, its payload is mathematically suppressed by the honest majority. However, by applying the scaling factor $\alpha = d+1$, the adversary precisely anticipates and neutralizes this structural dilution.

Prior to broadcasting its local parameters to its topological neighbors, the malicious client (denoted as node m) artificially scales the parameter weight differences using this topologically-derived boosting factor:

$$\mathbf{w}_{p,t}^{m,\text{boosted}} = \mathbf{w}_{p,t-1}^{m} + \alpha \cdot \left(\mathbf{w}_{p,t}^{m,\text{local}} - \mathbf{w}_{p,t-1}^{m} \right) \tag{4.2}$$

Where $p \in \{1, 2, \ldots, P\}$ denotes a specific parameter index within the total network weight space P, and t represents the current communication round. Here, $\mathbf{w}_{p,t-1}^{m}$ represents the baseline parameter value maintained by the attacker at the beginning of the iteration (acting as the localized reference model before optimization, effectively replacing the centralized "global" concept), $\mathbf{w}_{p,t}^{m,\text{local}}$ is the parameter value obtained after local training on the poisoned dataset, and $\mathbf{w}_{p,t}^{m,\text{boosted}}$ represents the final amplified parameter value prepared for dissemination.

During the neighborhood aggregation phase of the honest peers, the $\alpha = d+1$ boosting multiplier applied by the attacker is perfectly canceled out by the $\frac{1}{d+1}$ uniform aggregation weight applied by the receiver. This mathematical cancellation forces the backdoor payload directly into the local consensus while protecting the attacker's own neural network from severe parameter degradation, ensuring that the injected update remains structurally compatible with benign client contributions.

The utilization of this specific $\alpha = d + 1$ heuristic is not merely an optimization for the attacker, but a crucial control mechanism within our experimental design. In order to accurately evaluate the impact of network topology on backdoor propagation, the attack mechanism itself must be mathematically controlled.

If a random or static boosting factor was used in the attacks, the resulting success or failure of the attack could be mistakenly attributed to the arbitrary strength of the payload rather than the structural constraints of the graph. By deploying a dynamic heuristic that precisely neutralizes the mathematical dilution of any given neighborhood, we isolate the topological layout as the sole independent variable. This ensures that any variation in the attack's propagation speed or its ability to evade detection is strictly a consequence of the network's architecture.

4.3 Attacker Placement Strategy

As established in our threat landscape (Section 4.3), an adversary's effectiveness in DFL is fundamentally dictated by their position within the network graph. To systematically evaluate the resilience of DFL architectures, our methodology involved strategically placing the compromised node across distinct topological scenarios.

Rather than assigning the attacker randomly, we designed these placements to test the mathematical extremes of network connectivity. This approach isolates how different topological features natively accelerate or mitigate the spread of a continuous backdoor attack:

- **Fully Connected:** In this baseline scenario, the network graph is complete, meaning the attacker's placement is structurally symmetric to all honest nodes. The methodological objective here is to evaluate the backdoor's survival in an environment with absolute maximum density, where rapid, unimpeded peer-to-peer dissemination is possible, but where the honest consensus is also immediately unified.
- **Ring:** Representing the opposite extreme of the Fully Connected graph, the attacker is placed in a strictly sparse, closed loop where every node possesses a topological degree of exactly $d = 2$, meaning the attacker placement is also structurally symmetric. This topology is designed to evaluate how sequential communication naturally delays the malicious payload, forcing the backdoor to survive multiple hops of dilution without the aid of cross-network shortcuts.
- **Star:** This scenario isolates the vulnerabilities of localized centralization within a peer-to-peer framework. By systematically evaluating an attack originating from an isolated edge (a Peripheral Attacker) versus an attack originating from the highly connected center (the Hub Attacker), we test the resilience of the "consensus wall", a structural threshold where the clean parameters aggregated at a high-degree node completely overpowers and dilutes a single outlier update. This placement strategy determines whether a high-degree node serves as an effective structural barrier that neutralizes anomalies via neighborhood dilution, or functions as a highly connected distribution vector that accelerates adversarial propagation across the network state.
- **Stochastic Block Model:** To simulate realistic community-based networks, we evaluate two distinct attacker placements, distinguishing between a Peripheral Attacker (embedded in a local cluster) and a Gateway Attacker (positioned at the bridging link). First, we embed the attacker deep within a dense local cluster to test if the sparse gateway link natively functions as a topological firewall. Second, the attacker assumes the role of the bridging node itself. This comparative placement is specifically

engineered to determine whether the theoretical "dilution effect" can still contain a malicious payload when the adversary directly controls the structural bottleneck between two communities.

By deploying the $\alpha = d+1$ topological boosting heuristic across these carefully selected placements, this methodology ensures that the attack's effectiveness is evaluated not just by the strength of its mathematical payload, but by its ability to physically navigate the structural constraints of the decentralized graph.

4.4 Evaluation Framework: The Utility-Security Trade-off

When evaluating the resilience of a DFL system, it is not enough to simply measure how effectively an attack spreads. In a decentralized environment, the topological connections that allow a backdoor to propagate are the exact same connections that nodes rely on to improve their model's performance through collaboration. Because of this inherent conflict, our methodology is structured into two distinct evaluation phases to capture the complete picture:

- **The Non-Adversarial Baseline (Measuring the Collaboration Gain:)** Before introducing any malicious activity, we first evaluate the performance of each network topology in a purely benign environment. This phase establishes the Collaboration Gain, quantifying how much each node's model performance improves through decentralized collaboration alone. By measuring the Clean Accuracy of each node after multiple rounds of training and aggregation, we can identify the actual improvement in accuracy and optimization stability that a node achieves by communicating with its peers instead of learning in strict isolation. This phase essentially proves why the nodes are taking the risk of collaboration in the first place, and sets a benchmark for the maximum potential utility that can be achieved in each topology.

- **The Adversarial Evaluation (Measuring the Security Risk):** Once we establish the structural value of a topology, we introduce the targeted backdoor attack defined in Section 4.2. This phase evaluates the security risk of those peer-to-peer connections by quantifying the Attack Success Rate (ASR). Formally, ASR measures the operational effectiveness of the backdoor payload and is defined as the proportion of trigger-patched test samples that are successfully misclassified into the adversary's malicious target class:

$$\text{ASR}_i = \frac{1}{|\mathcal{D}_{\text{test}}^{\text{trigger}}|} \sum_{x \in \mathcal{D}_{\text{test}}^{\text{trigger}}} \mathbb{I}\left(f(x; \mathbf{w}_i) = y_{\text{target}}\right) \tag{4.3}$$

where $\mathcal{D}_{\text{test}}^{\text{trigger}}$ represents the validation dataset where every clean sample has been modified with the visual trigger patch, $f(x; \mathbf{w}_i)$ denotes the classification output of node i's local model parameterized by weights $\mathbf{w}_i$, y_{target} is the attacker's target label (in our setup, Class 0) and $\mathbb{I}(\cdot)$ is the indicator function that outputs 1 if the condition is satisfied and 0 otherwise. Tracking how this metric behaves across honest nodes when the adversary applies the degree-scaled boosting heuristic allows

us to systematically chart the speed, depth, and boundaries of adversarial propagation within each network geometry.

CHAPTER 5

Experimental Design and Setup

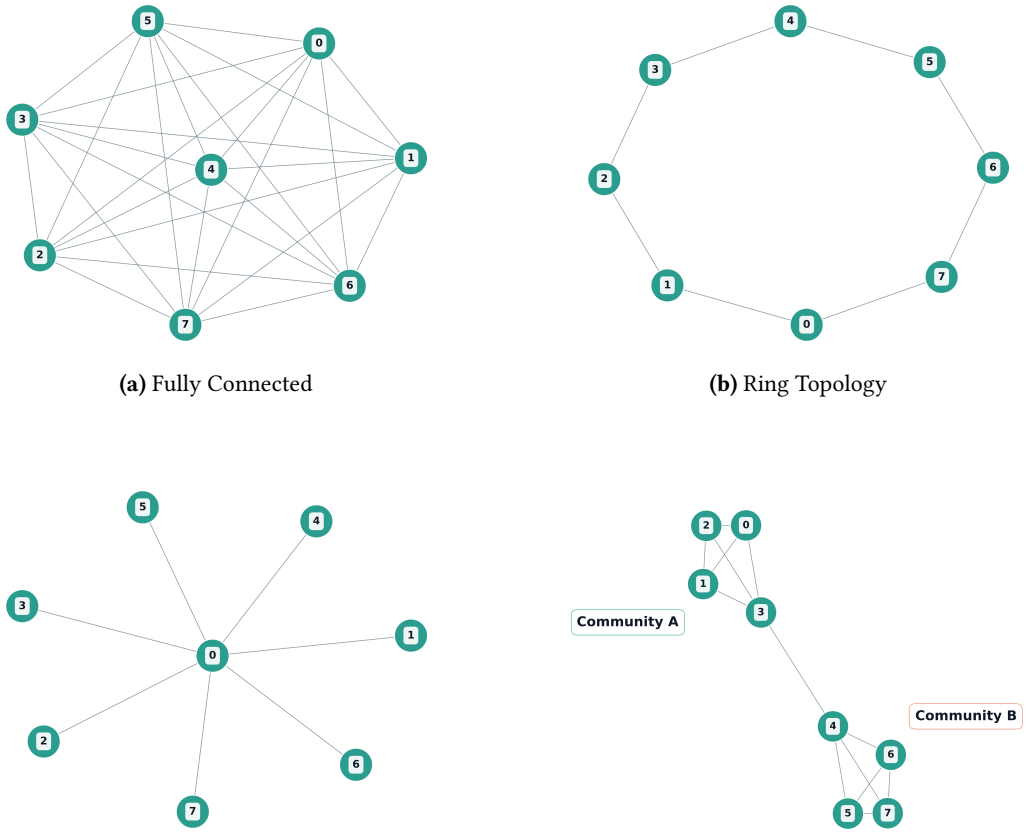
This chapter details the experimental framework deployed to evaluate decentralized network resilience against targeted backdoor exploits. By simulating continuous poisoning attacks across diverse graph architectures, this setup empirically isolates how different communication geometries naturally accelerate or impede malicious parameter propagation. All experiments were conducted using a custom Flower environment [32, 33]. The source code and configuration files used to reproduce all the experiments are publicly available at: https://github.com/IkerMardure/tfg-gossip-learning-backdoor

5.0.1 The Flower Federated Learning Framework

To implement and evaluate the proposed decentralized backdoor attacks, all experiments were conducted using a highly customized environment built upon the Flower (`Flwr`) framework [32]. Flower is a widely adopted, open-source framework designed for FL at scale. It abstracts the complex low-level mechanics of distributed communication, allowing researchers to focus on algorithmic design while providing robust tools for simulating thousands of clients on limited hardware through its Virtual Client Engine.

However, it is crucial to note that Flower natively operates under a strict, centralized client-server architecture (e.g., standard FL). Because this thesis explicitly investigates purely DFL, the standard Flower orchestrator could not be directly used.

To bridge this gap, we utilized GLow [33], a simulation strategy built upon Flower that leverages its robust node abstraction and local training execution while fundamentally overriding its global aggregation logic. Instead of clients sending updates to a central server, GLow forces the framework to simulate peer-to-peer parameter diffusion based on predefined graph topologies. This architectural adaptation by Belenguer et al. was essential to our project: it allowed us to harness Flower's efficient PyTorch integrations and hardware resource management, while successfully simulating the exact topological constraints and neighborhood averaging. It also allowed us to implement the adversarial boosting required to evaluate decentralized backdoor propagation.



(a) Fully Connected

(b) Ring Topology

(c) Star Topology

(d) Stochastic Block Model

Figure 5.1: Visualization of the four network topologies evaluated in this study.

5.1 Experimental Network Instantiation

While the theoretical properties and adversarial placement objectives for each graph configuration were established in Section 4.3, this section details their concrete technical instantiation within our simulation framework. To isolate network geometry as the sole independent variable, all evaluated topologies are modeled as static graphs $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ containing a uniform scale of $|\mathcal{V}| = 8$ participating clients. This controlled size allows for a precise, step-by-step observation of localized parameter dilution.

The four architectures are visualized in Figure 5.1 and are instantiated deterministically within the Flower environment using the following exact topological parameters:

- **Fully Connected Topology:** Instantiated as a complete graph where every node maintains a degree of $d_i = 7$, establishing a network diameter of exactly one hop and maximizing consensus drag.
- **Ring Topology:** Configured as a uniform closed loop where every client is strictly bounded to a topological degree of $d_i = 2$, forcing multi-hop sequential propagation.
- **Star Topology:** Structured as a highly asymmetric central-hub framework, consisting

of one central coordinator node with a degree of $d_0 = 7$ and seven peripheral leaf nodes restricted to $d_i = 1$.

- **Stochastic Block Model (SBM):** Implemented as a deterministic two-community graph. Community A (Nodes 0–3) and Community B (Nodes 4–7) are internally complete clusters. The sub-graphs are bridged exclusively by a single, static inter-community edge linking Node 3 and Node 4, assigning them a degree of $d_3 = d_4 = 4$, while the remaining peripheral nodes maintain $d_i = 3$.

This structural mapping provides the experimental infrastructure required to evaluate the utility-security trade-off detailed in the following sections.

Deterministic SBM Implementation

It is important to note that while the theoretical Stochastic Block Model [34] is a probabilistic framework relying on random edge generation, our experimental methodology requires a controlled, deterministic environment to accurately isolate and measure the attack's propagation.

As shown in Figure 5.1d, the implemented network consists of two distinct communities containing three nodes each. Within each community, the nodes are fully connected to ensure rapid local consensus. However, the two communities are bridged by exactly one fixed gateway connection. To execute our comparative placement strategy (as defined in Section 4.3), we evaluate two distinct attack scenarios within this structure:

- **Scenario 1 (Peripheral Attacker):** The designated malicious node is Client 1, embedded deep within Community A.
- **Scenario 2 (Gateway Attacker):** The designated malicious node is Client 2, which acts as the direct structural bottleneck bridging Community A and Community B.

This deterministic design guarantees that we can directly quantify the natural firewall capabilities of a community boundary versus the potential of a compromised structural bottleneck.

5.2 Experimental Setup: Dataset, Architecture, and Metrics

The primary experiment was conducted using the MNIST dataset [10]. To align with the architectural requirements of the model, all images were resized to 32×32 pixels. The classification task used all 10 classes of the MNIST dataset. To establish a controlled and uniform baseline, the global dataset was evenly divided into strictly disjoint partitions and distributed in an Independent and Identically Distributed (IID) manner among the clients. This guarantees that each node is assigned an equally sized, mutually exclusive, and statistically similar subset of the training data, preventing any sample overlap between peers.

A modified LeNet [10] architecture was used as the local model across all nodes. The network is composed of three convolutional layers of 16, 32, and 64 filters, respectively, each with 3×3 kernels and a padding of 1. Each convolutional operation is followed by a ReLU activation function and a 2×2 Max Pooling operation. The resulting feature maps

are processed through two fully connected layers, the first containing 128 units and the last one containing as many units as the classes used (in this case, 10). A dropout layer with a 0.2 probability was also inserted prior to the final classification layer to mitigate overfitting during the training.

To accurately evaluate the long-term assimilation of the continuous backdoor attack, the global training process spanned 240 communication rounds. The first 16 communication rounds were executed under strictly non-adversarial conditions, establishing a brief warm-up phase. This configuration allows the decentralized local models to initially synchronize and begin converging on legitimate data features before facing adversarial interference. Following this stabilization period, the continuous poisoning attack is actively triggered from round 16 onward. During each subsequent round, the active clients performed local training for 2 epochs using a batch size of 128. This extended timeframe is essential for observing the slow, steady displacement of the global performance.

To evaluate the attack, two primary metrics were used: Clean Accuracy and Attack Success Rate (ASR). Clean Accuracy measures how well each node classifies the unmodified test data, which is critical for observing how undetectable (stealthy) the attack remains during normal operation. In contrast, the ASR is calculated using modified test data to indicate the spread and effectiveness of the backdoor payload through the network.

Simulation Constraints and Execution Flow: While the theoretical framework of GL often assumes fully parallel, asynchronous execution across all participating nodes (as outlined in Algorithm 2.2), simulating such a decentralized network on centralized hardware requires specific implementation constraints.

In our experimental setup using the Flower framework [32], the execution flow was implemented as a discrete-event sequential simulation rather than a truly parallel asynchronous environment. Specifically, a single communication round is strictly defined as the local training and subsequent parameter broadcast of exactly one client. During its designated round, the active client performs local optimization and immediately sends its updated model to its topological neighbors, who aggregate it into their own local state. Consistent with the standard decentralized protocols established in our threat model, this aggregation applies a uniform topological weighting; the incoming model is scaled by a factor of $\alpha_{ij} = \frac{1}{d_i+1}$, meaning all neighboring updates are treated equally based on the receiving node's topological degree.

This discrete-event approach provides a highly controlled baseline for evaluating topological flow. By isolating the parameter dissemination from the unpredictable network noise, latency, and packet drops inherent to real-world peer-to-peer environments, we can precisely track the mathematical dilution of the backdoor payload.

To ensure a fair and balanced evaluation, it was imperative that every node in the network received an equal amount of training exposure. Therefore, the total number of communication rounds was scaled linearly based on the network size to achieve 30 full network training cycles. A complete breakdown of the hardware specifications, software frameworks, and runtime metrics associated with these simulations is detailed in Appendix B.

5.3 Industrial Validation Setup: NEU-DET

To bridge the gap between theoretical insights and practical applicability, a secondary experimental setup was implemented using the NEU-DET dataset [11]. This dataset consists of 1800 grayscale images of size 200×200 pixels, categorized into six classes representing different types of surface defects in steel manufacturing. The classification task is to identify the specific type of defect present in each image, providing a complex, real-world scenario for evaluating the resilience of decentralized networks. Consistent with the primary MNIST baseline, the global dataset was evenly divided into strictly disjoint partitions and distributed in an Independent and Identically Distributed (IID) manner. This ensures that the structural evaluation remains unaffected by data redundancy between peers.

To accommodate the increased visual complexity of industrial defects, a more sophisticated architecture was implemented. The local model is based on a ResNet-18 [35] architecture, which is well-suited for capturing intricate spatial features. The network was modified to accept single-channel grayscale images and to output six classes corresponding to the defect types. Similar to the MNIST setup, all parameters were set to be trainable to allow the backdoor attack to effectively learn the new visual trigger.

The input images were scaled to 128×128 pixels to balance computational efficiency with the need for sufficient resolution to capture defect details. The visual backdoor trigger was also scaled to a 28×28 white patch to maintain a consistent relative size compared to the original MNIST trigger (see Figure 5.2).

To manage the increased memory overhead of the ResNet-18 architecture, the training batch size was adjusted to 16, and the learning rate was calibrated to 0.0001 to ensure stable convergence during the 240 communication rounds. The same metrics of Clean Accuracy and Attack Success Rate were used to evaluate the attack's performance and stealthiness in this more complex, real-world scenario.

Experimental Scenarios: The NEU-DET validation specifically targeted the two structural extremes evaluated in our theoretical framework: the Fully Connected topology and the Stochastic Block Model. The Fully Connected topology serves as a stress test for the attack's maximum potential, while the Stochastic Block Model provides insights into how community structures can naturally impede the spread of malicious payloads in real-world industrial networks.

By utilizing the same continuous $\alpha = d + 1$ topological heuristic, this industrial validation isolated the network topology as the primary variable, demonstrating that the structural vulnerabilities identified in the baseline experiments are inherent to DFL and not merely artifacts of simplified datasets or architectures.

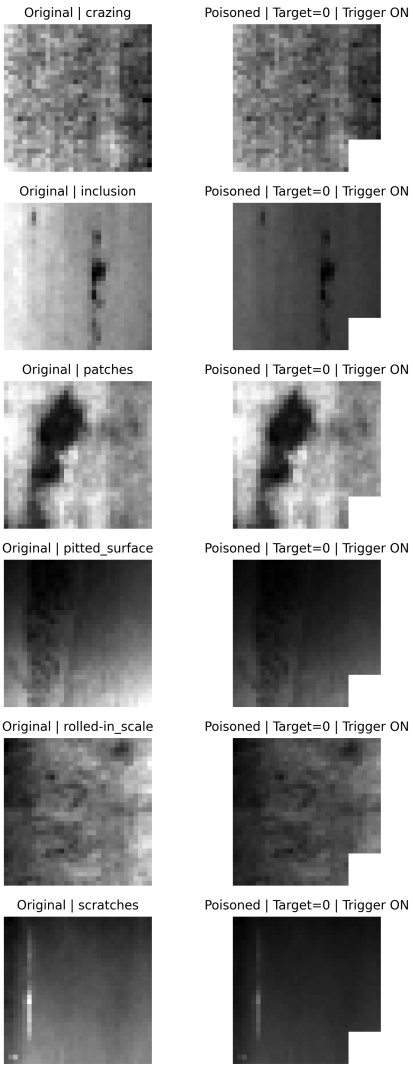


Figure 5.2: Examples of clean and poisoned (triggered) industrial surface defect images from the NEU-DET dataset.

CHAPTER 6

Experimental Results and Discussion

This chapter presents the experimental results and evaluates the behavior of the decentralized learning system under a targeted backdoor attack. The analysis focuses on how different network architectures affect both the learning of legitimate data and the propagation of the malicious infection. The evaluation is divided into several phases: establishing the baseline resilience of standard topologies, analyzing the attacker's trade-off between effectiveness and stealthiness, evaluating community-based structures (SBM), and testing the system under scaled and more realistic conditions. For comprehensive visualizations of the individual node performances across all these scenarios, please refer to Appendix A.

6.1 The Collaboration Gain: Establishing the Non-Adversarial Baseline

Before introducing the adversarial threat, it is essential to quantify the fundamental value of DFL in a non-adversarial context. *Why do nodes take the risk of exposing their models to a peer-to-peer network?* To answer this, the network was evaluated under clean, honest conditions for 240 communication rounds using 8 clients. This extended timeframe allows us to observe the true convergence behavior and the long-term benefits of collaboration of each topology. See individual node metrics in Figures A.1, A.2, and A.3 in Appendix A for a complete view of the performance variance and loss evolution across all clients.

The Cost of Isolation: As a control baseline, the clients were first evaluated training in strict isolation (no communication). The results in Figure 6.1 clearly illustrate the limitations of purely local training. The Distributed Loss exhibits high variance, fluctuating significantly throughout the 240 rounds. Furthermore, the Clean Accuracy shows significant variance between individual clients. This high noise indicates that isolated nodes overfit rapidly to their limited local data, constantly struggling to reach a stable minimum without external regularization.

Fully Connected Topology: When the clients are allowed to communicate without restrictions, the network performance transforms into a smooth, monotonic exponential

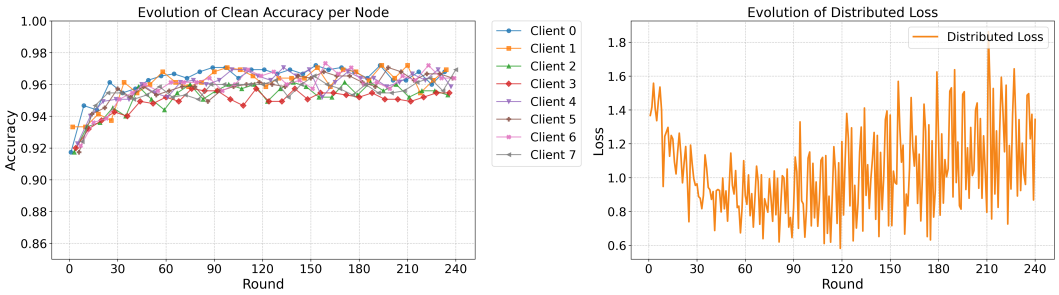


Figure 6.1: Clean Accuracy and Distributed Loss over 240 rounds for the Isolated (No Communication) topology.

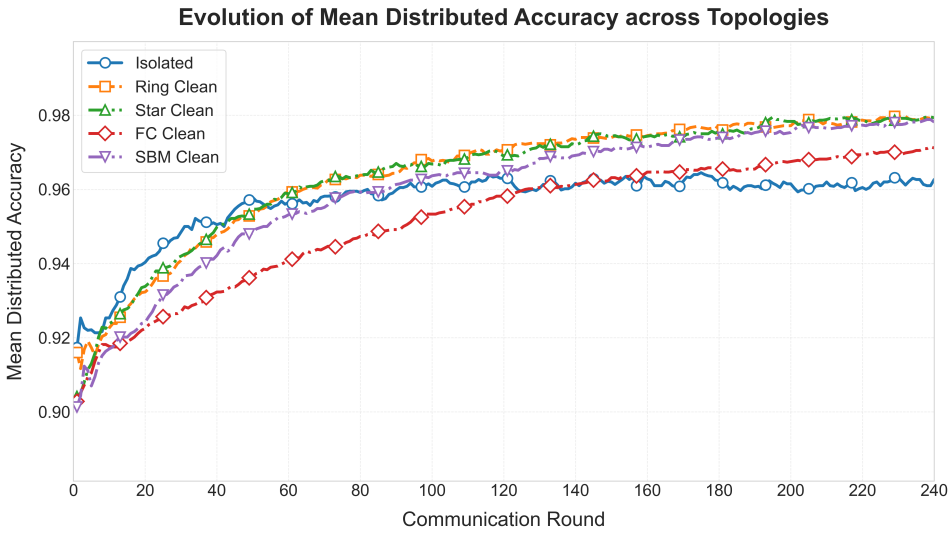


Figure 6.2: Evolution of Mean Distributed Accuracy across all evaluated topologies over 240 communication rounds.

climb, as seen in the aggregate accuracy curve (Figure 6.2). The constant averaging acts as a powerful regularizer, effectively filtering out the stochastic noise seen in the isolated nodes. However, because the execution is asynchronous, this maximum density also introduces a severe Staleness Penalty [36], a phenomenon where aggregating out-of-date local models causes unexpected turbulence and degrades convergence speed. Each node averages its fresh updates with seven other peers, many of which hold stale parameters from previous rounds. This chronic over-averaging acts as a consensus drag, preventing the network from descending the loss landscape as deeply as sparser topologies, resulting in a final loss around 0.54 (see Table 6.1).

Ring Topology: When structural constraints are introduced, the physical shape of the network becomes a massive optimization advantage in an asynchronous environment. As demonstrated in Figure 6.2, the Ring topology breaks past the consensus drag of the denser network, achieving a higher final mean accuracy and a significantly lower final loss of approximately 0.32 (Table 6.1). Because a node only averages its weights with two neighbors ($d = 2$), the Ring topology naturally minimizes the staleness penalty and finds a way to take advantage of the communication efficiency.

Star Topology: The Star topology demonstrates the raw efficiency of localized cen-

Topology	Final Loss	Final Accuracy (Mean)	Rounds to 96% Acc.
Isolated	1.10 ± 0.15	0.963	N/A
Fully Connected	0.54	0.971	[96 - 227]
Ring	0.32	0.980	[52 - 130]
Star	0.26	0.979	[49 - 85]
Stochastic Block Model	0.41	0.975	[61 - 140]

Table 6.1: Summary of non-adversarial baseline metrics at Round 240. For the "Rounds to 96% Acc." metric, the brackets $[x - y]$ denote the communication round at which the first node (x) and the last node (y) in the network achieved a Clean Accuracy of 96%. This specific 96% threshold was selected as it represents a robust and stable convergence baseline for the LeNet architecture on the MNIST dataset, providing a clear reference point to compare the communication efficiency across different topologies. N/A (Not Applicable) indicates that isolated nodes failed to stably maintain this threshold due to persistent oscillation and high variance. The standard deviation for the final loss is explicitly reported only for the isolated baseline to highlight the severe architectural divergence caused by the lack of communication; collaborating topologies achieve tight network consensus, rendering their individual variance negligible.

tralization in an asynchronous environment, achieving the fastest convergence (Figure 6.2) and the lowest final loss of 0.26 (Table 6.1). This exceptional performance is due to its highly asymmetric distribution. The peripheral nodes, which make up the majority of the network, possess a topological degree of only one, so they retain 50% of their own optimized parameters during each round, suffering minimal staleness while also benefiting from the communication. While the central hub (Client 0) absorbs the collective staleness of the entire network, it acts as a highly refined global regularizer for the peripherals. This unique dynamic allows the Star topology to achieve a powerful balance between local optimization and global consensus, resulting in superior convergence behavior.

SBM Topology: Finally, the SBM topology acts as a clear mathematical middle ground for our staleness hypothesis. Its final accuracy and loss stabilized exactly between the sparse Star/Ring topologies and the dense Fully Connected network (Figure 6.2 and Table 6.1). This empirically shows that the staleness penalty scales with local network density. The SBM nodes suffer from local consensus drag within their internal communities ($d = 3$), but the structural bottleneck effectively shields them from the massive, network-wide over-averaging that restricts the Fully Connected topology and it accomplishes a good balance between isolation and collaboration.

6.2 Baseline Topologies Resilience

The second phase of the evaluation analyzed the propagation of the continuous backdoor attack across three fundamental communication structures: Fully Connected, Ring, and Star. To accurately measure the long-term assimilation of the payload, the network consisted of 8 nodes operating over 240 communication rounds. The adversary (Client 1) employed the optimized stealth configuration defined in Chapter 4: a 20% local poisoning ratio and the topological boosting heuristic of $\alpha = d + 1$.

Before analyzing the specific structural mechanics, Figure 6.3 provides a macroscopic

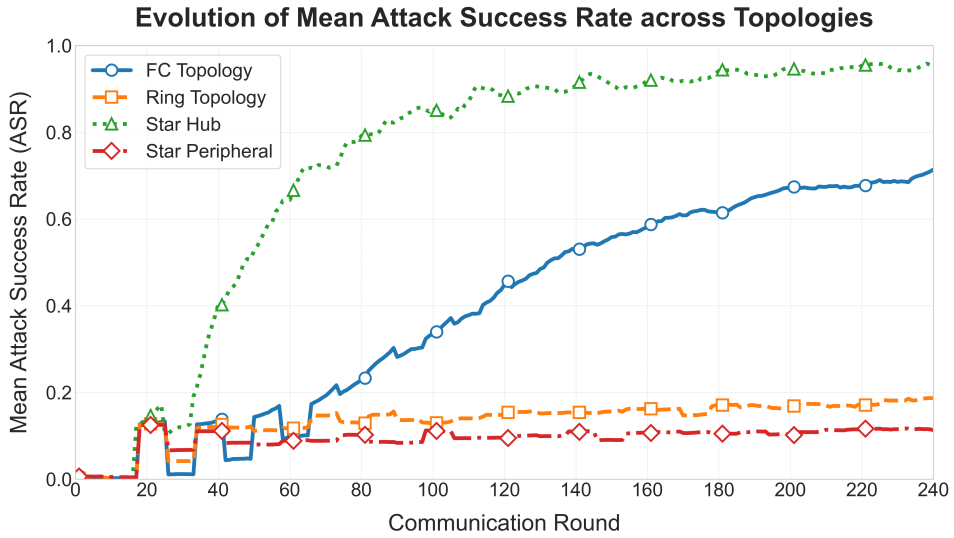


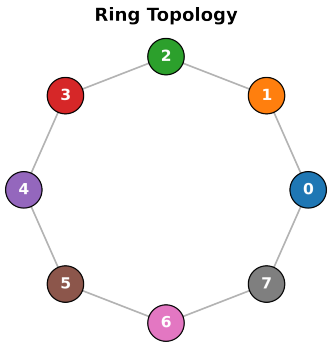
Figure 6.3: Evolution of Mean Attack Success Rate (ASR) across baseline topologies over 240 communication rounds.

comparison of the global Attack Success Rate (ASR) across baseline architectures. This mean evaluation clearly illustrates the extremes of decentralized vulnerabilities: the highly connected Star Hub attacker completely dominates the network, while the Peripheral and Ring configurations naturally suppress the payload's propagation. However, understanding exactly how these topologies accelerate or trap the attack requires examining the individual node behaviors.

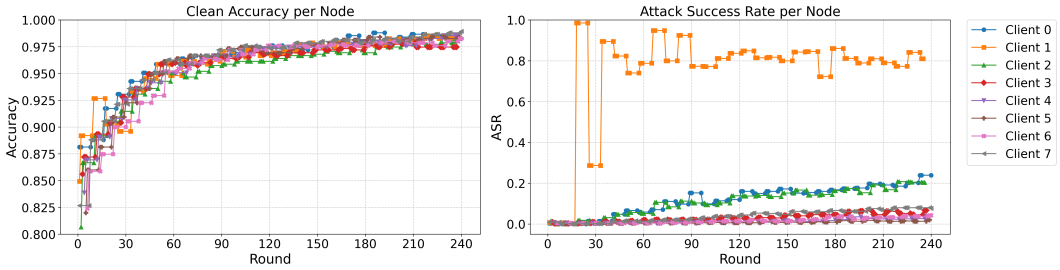
Fully Connected Topology: As demonstrated by the mean ASR curve (Figure 6.3), the dense Fully Connected network lacks the communication barriers necessary to halt the infection. While the honest nodes successfully resisted the backdoor for the first 30 rounds, the continuous injection strategy eventually overwhelmed them. Starting around round 45, the mean ASR begins a steady climb, eventually surpassing 0.70 by round 240. Because the topology is perfectly symmetrical, all honest nodes behave identically, meaning this mean plot perfectly represents the individual node experience. This demonstrates that in highly connected networks, the $\alpha = d + 1$ heuristic effortlessly overcomes neighborhood dilution, resulting in a system-wide infection. See Figure A.4 in Appendix A for the complete individual node metrics, which illustrate the uniformity of the infection across all honest clients.

Ring Topology: The Ring topology demonstrated a strong structural resistance to rapid propagation. While the mean ASR remains low, the individual node performance in Figure 6.4b reveals the exact mechanics of this defense. Due to the strict neighborhood constraints ($d = 2$), the backdoor was forced to travel sequentially, creating a clear staggered infection. The attacker's immediate neighbors (Clients 0 and 2) slowly climbed to an ASR of approximately 0.25 by the end of the simulation. However, the distant nodes (Clients 3 and 4) were largely shielded by the topological distance, maintaining an ASR below 0.20 for over 90 rounds. The structural bottleneck of the Ring effectively trapped the infection, proving that sparsity natively buys the network time against continuous poisoning.

Star Topology: As highlighted by the extreme divergence in the global mean ASR



(a) Ring Topology



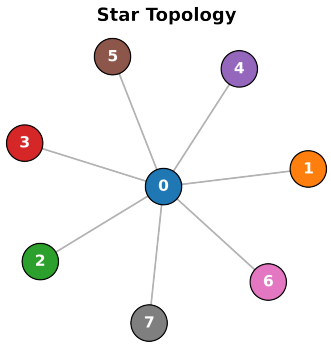
(b) Ring Performance

Figure 6.4: Evaluation of the Ring topology over 240 rounds. The staggered node infection is visually evident in the performance plot, corresponding to the topological distance from the attacker (Client 1).

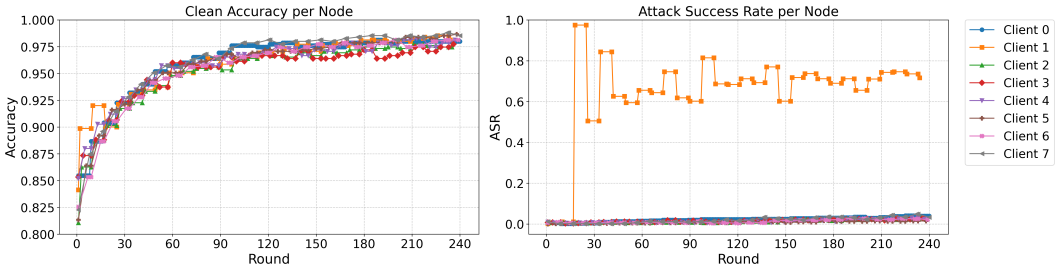
(Figure 6.3), the Star topology represents both the most secure and the most vulnerable architecture depending on the adversary's position. To fully understand the mechanics of localized centralization within a peer-to-peer framework, the topology was evaluated under two distinct attacker placements: an attack from an isolated edge (Peripheral Attacker) and an attack from the structural center (Hub Attacker).

When the adversary is placed at the periphery (Figure 6.5b), the topology exhibits significant structural robustness. The attacker (Client 1) attempts to push its backdoor, but the central hub (Client 0) continuously averages the poisoned weights with clean updates from the other isolated nodes. This central aggregation acts as a highly resilient structural barrier, effectively neutralizing the malicious parameters through local neighborhood dilution. The ASR for the hub and all honest peripheral nodes remains flat near 0.0 for the entire simulation.

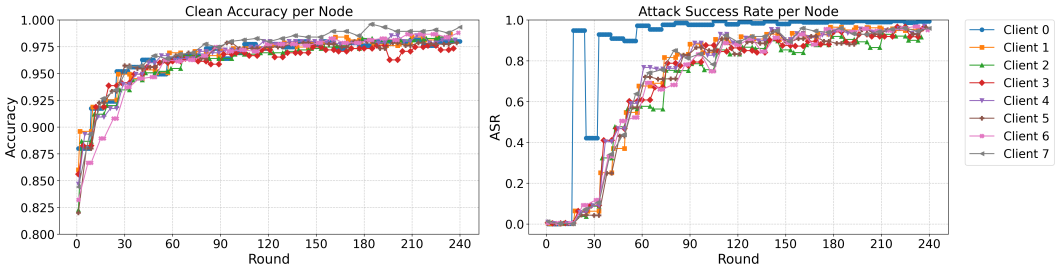
However, when the adversary compromises the central Hub itself (Figure 6.5c), the network becomes fragile. Because Client 0 connects to all nodes and the peripheral nodes have no lateral connections to one another, the attacker possesses a direct, unfiltered broadcast channel. As demonstrated by the ASR graph, the attacker quickly embeds the backdoor into its local model and systematically transmits it. Without any alternative honest neighbors to dilute the payload, the peripheral nodes are defenseless. By round 60, the ASR for all clients surges past 0.60, resulting in the devastating network-wide compromise previously observed in the global comparison, while the attacker perfectly



(a) Star Topology



(b) Star Topology — Peripheral Attacker



(c) Star Topology — Hub Attacker

Figure 6.5: Evaluation of the Star topology under two distinct attacker placements. The node colors in the top architectural map correspond strictly to the client lines in the performance plots.

preserves its baseline utility.

These baseline results definitively prove that an attack's effectiveness is not strictly tied to the payload itself, but is also governed by the architectural constraints of the decentralized graph.

6.3 Navigating the Effectiveness vs. Stealthiness Trade-off

A crucial observation across all evaluated topologies is the definitive resolution of the effectiveness versus stealthiness trade-off. Traditionally, high-intensity model poisoning, characterized by aggressive boosting factors and high local poisoning ratios, risks causing severe parameter degradation. In such scenarios, the excessive parameter distortion causes

the local model's Clean Accuracy to collapse, introducing uncontrolled parameter variance that ruins the experimental baseline.

However, the implementation of our proposed Continuous Poisoning strategy (utilizing a 20% poison ratio) combined with the topologically-derived boosting heuristic ($\alpha = d + 1$) successfully overcomes this inherent limitation. As demonstrated in the individual node performance plots across this chapter (see the Clean Accuracy subplots in Figures 6.4, 6.5, and Appendix A), the Clean Accuracy of the malicious node remains consistently above 0.95, closely mirroring the performance of the honest majority.

By carefully pacing the injection across 240 rounds and scaling the payload mathematically to the network's local dilution rate, the adversary successfully avoids severe parameter degradation. This configuration ensures that the malicious updates remain structurally compatible with benign client contributions, preserving a stable and controlled baseline to evaluate the structural topology as the sole independent variable governing network-wide backdoor propagation.

6.4 Community Topologies and Boundary Resilience (SBM)

To evaluate the resilience of realistic, community-based networks, we simulated an 8-node Stochastic Block Model (SBM) divided into two distinct communities (Community A: Clients 0-3; Community B: Clients 4-7) connected by a single gateway link between Client 3 and Client 4. The simulation spanned 240 rounds to test long-term boundary resilience.

6.4.1 The Topological Firewall (Peripheral Attacker)

In the first scenario (Figure 6.6b), the attacker is placed at Client 1, deep within Community A. The results demonstrate a rapid intra-community amplification. Because Client 1 is well-connected locally but isolated from the broader network, the backdoor successfully circulates within Community A, pushing the ASR of its immediate neighbors (Clients 0 and 2) to approximately 0.45 by round 240.

However, the attack entirely fails to cross the community boundary. The ASR for all nodes in Community B (Clients 4, 5, 6, and 7) remains flat at 0.0 for the entire simulation.

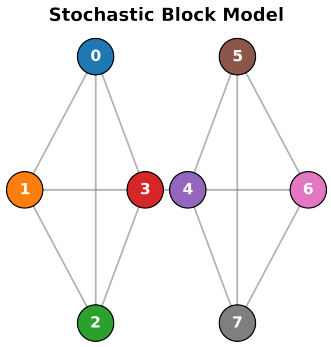
This proves that a sparse gateway link natively functions as a physical, topological firewall. Community B is too dense for a peripheral infection to penetrate via a single bridging connection.

6.4.2 The Bottleneck Breach Attempt (Gateway Attacker)

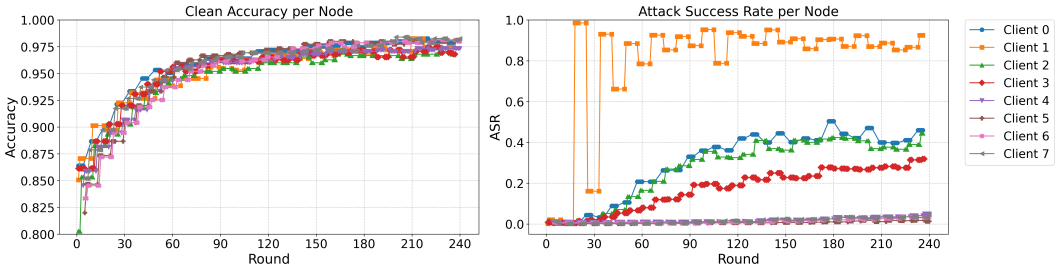
To push the network to its limit, the final experiment shifted the compromised node to the structural bottleneck itself. In this scenario (Figure 6.6c), Client 3 (the gateway node of Community A) acts as the adversary, utilizing a higher dynamic boosting factor ($\alpha = d + 1$) commensurate with its higher connectivity.

The change in attacker placement yielded two meaningfull discoveries:

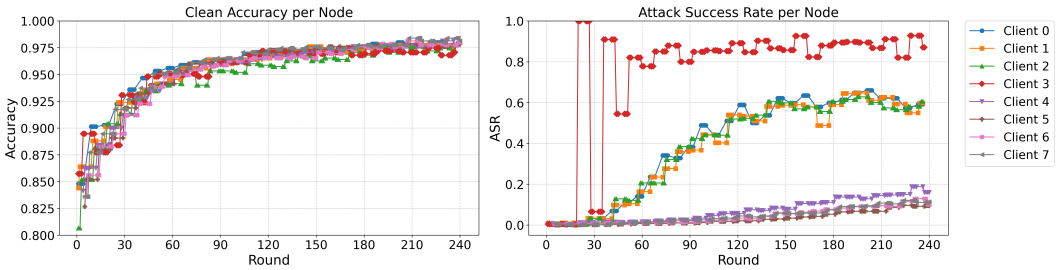
- **Amplified Local Infection:** Because the attacker was positioned at a highly central point within Community A, the local infection rate increased substantially. Clients 0, 1, and 2 experienced an ASR surge to roughly 0.60, which is significantly higher than



(a) Stochastic Block Model Topology



(b) SBM — Peripheral Attacker



(c) SBM — Gateway Attacker

Figure 6.6: Clean Accuracy and ASR over 240 rounds for the Stochastic Block Model. The top figure illustrates the network topology, with node colors matching the individual client lines in the subsequent plots under two distinct attacker placements.

the peripheral attack. Controlling the gateway allowed the adversary to disproportionately influence its own community.

- **Global Mitigation via Honest Consensus:** Despite the attacker having a direct edge to Community B and continuously broadcasting for 240 rounds, the broader network successfully contained the infection. The adversarial payload failed to establish dominance in the neighboring cluster, as the ASR across clients 4 through 7 was heavily suppressed, plateauing below 0.15.

This ultimate failure to bridge the gap highlights the defensive power of DFL. When the gateway node of Community B (Client 4) received the highly poisoned parameters from

Topology	Attacker Placement	Local ASR	Global ASR	Clean Acc.
Fully Connected	Symmetric	0.673	0.673	0.9693
Ring	Symmetric	0.232	0.079	0.9827
Star	Peripheral (Edge)	0.038	0.023	0.9813
Star	Hub (Center)	0.9925	0.9587	0.9933
SBM	Peripheral (Comm A)	0.444	0.0354	0.9813
SBM	Gateway (Comm A)	0.5973	0.1287	0.9786

Table 6.2: Summary of adversarial backdoor metrics at Round 240. Local ASR refers to the infection rate of the attacker's immediate neighbors or local community, while Global ASR refers to distant nodes or secondary communities.

Client 3, it immediately aggregated them with the clean updates from its internal neighbors (Clients 5, 6, and 7).

Community B functioned as a large "consensus sink," naturally diluting the backdoor payload before it could gain any real traction.

These experiments prove that while continuous, topology-aware backdoor attacks can successfully maintain metric stability and dominate dense local clusters, decentralized community boundaries provide a natural, highly robust defense against global network corruption. A comprehensive breakdown of the final localized and global infection rates across all evaluated topologies is detailed in Table 6.2.

6.5 Validation on Industrial Data (NEU-DET)

To ensure that the observed topological vulnerabilities are not merely an artifact of the relatively simple MNIST dataset, the core experiments were replicated using the NEU-DET dataset. This dataset serves as a complex benchmark for industrial surface defect detection.

The objective was to verify whether the mechanics of decentralized backdoor propagation hold true in high-stakes engineering applications. In particular, we aimed to test the dissemination in dense networks and the firewall effect of sparse boundaries. The network was evaluated under two distinct structural extremes. The first is a dense Fully Connected topology, and the second is a community-based SBM with two attacker positions: a Peripheral Attacker and a Gateway Attacker. Across all configurations, the malicious node utilized the continuous $\alpha = d + 1$ topological boosting heuristic.

As observed in the left subplots of Figure 6.8, the baseline Clean Accuracy across all nodes exhibits higher variance and oscillation compared to the MNIST experiments. This behavior is expected when training a deeper architecture like ResNet-18 on a complex industrial dataset using decentralized, small-batch optimization. However, the attacker perfectly mirrors the accuracy curve of the honest majority. The malicious node maintains strict metric stability, ensuring it remains statistically indistinguishable from its peers despite the continuous injection of the 28×28 trigger payload.

Before analyzing the localized node behaviors, Figure 6.7 provides a macroscopic

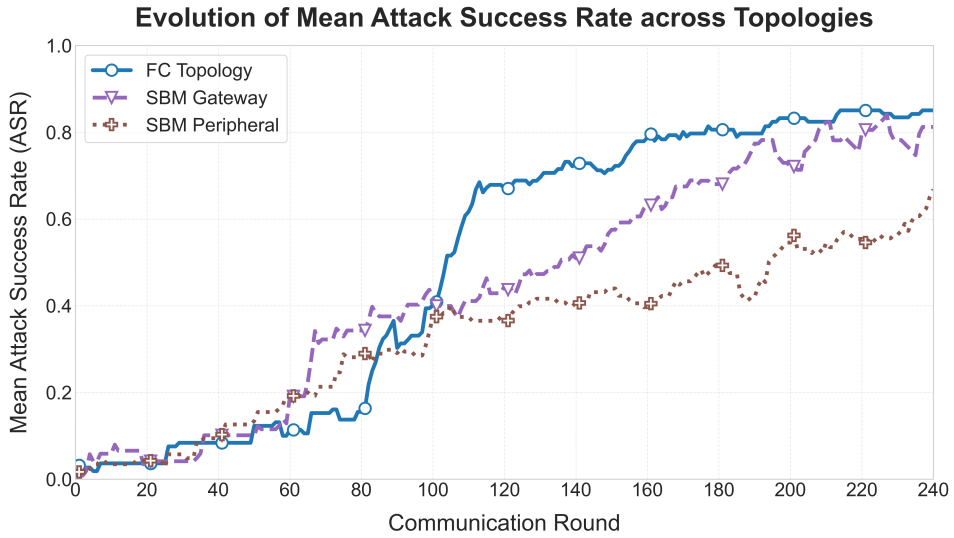


Figure 6.7: Evolution of Mean Attack Success Rate (ASR) across evaluated topologies on the NEU-DET dataset over 240 communication rounds.

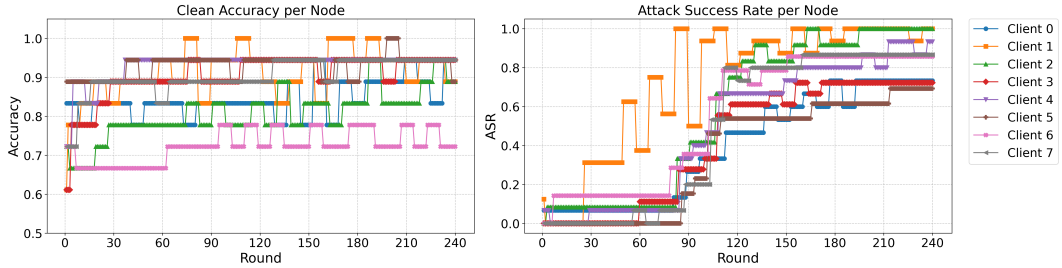
comparison of the Mean Attack Success Rate (ASR) across the three industrial scenarios. While metric stability was preserved, this global view reveals that dataset complexity alters the absolute boundaries of our topological defenses, shifting them from absolute firewalls to powerful temporal delays.

In the Fully Connected network (Figure 6.8a), the absence of communication bottlenecks allowed the continuous injection to systematically overcome neighborhood dilution. This lack of structural resistance drove a rapid, system-wide infection, with the ASR for all clients surging to between 0.70 and 1.0 by the end of the 240 rounds.

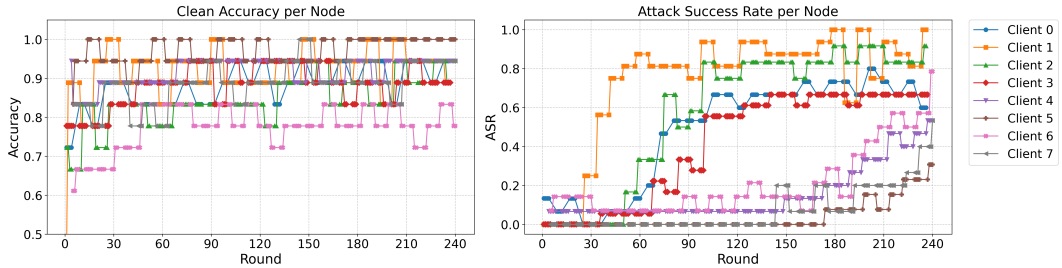
When the attacker was embedded deep within Community A of the SBM topology (Figure 6.8b), the network exhibited robust, long-term boundary resilience. The malicious payload successfully circulated within the local feedback loop, infecting the immediate neighbors (ASR $>$ 0.60 by round 90). However, the single gateway link acted as a formidable temporal firewall. Community B was completely shielded from the payload for over 170 communication rounds. While the continuous nature of the attack eventually caused the backdoor to leak across the boundary in the final stages of the simulation (reaching an ASR of 0.20 to 0.50), the structural bottleneck successfully delayed the global compromise by a massive margin.

To push the industrial validation to its limit, the adversary was repositioned to the gateway node itself (Figure 6.8c). Commanding this bottleneck amplified the local infection within Community A, pushing the ASR to nearly 1.0. Furthermore, the direct broadcast channel to Community B eventually overcame the consensus sink defense. While Community B successfully suppressed the payload for the first 120 rounds, the continuous, degree-scaled injection directly at the bridging link eventually overpowered the local dilution, allowing the ASR in the distant community to climb to roughly 0.60–0.80 by round 240.

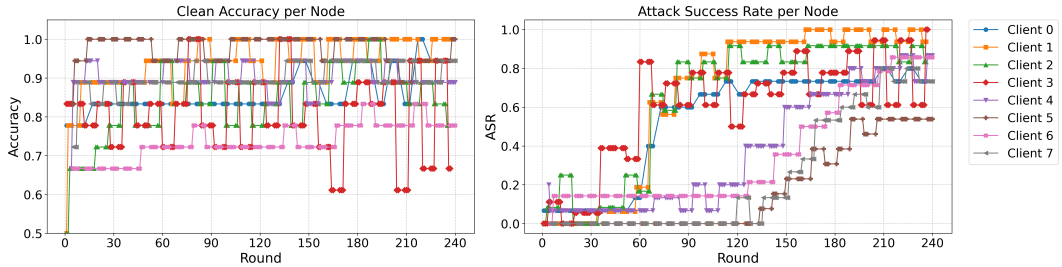
These industrial validations confirm a nuanced version of the core hypothesis. In high-complexity DFL environments, sparse community boundaries may not provide per-



(a) Fully Connected



(b) SBM — Peripheral Attacker



(c) SBM — Gateway Attacker

Figure 6.8: Clean Accuracy and ASR over 240 rounds for the NEU-DET dataset under three distinct topological configurations.

manent immunity against a continuous, mathematically scaled adversary, but they act as highly effective structural delays. By trapping or suppressing the payload for hundreds of communication rounds, the topology buys the network critical time to deploy active intervention protocols before global corruption occurs.

CHAPTER 7

Conclusions and Future Work

7.1 Summary of Contributions

While the shift from centralized to DFL successfully eliminates the risks of a single point of failure and centralized data monopolies, it places the burden of security squarely on the network's structural topology. This thesis systematically evaluated the structural vulnerabilities of these peer-to-peer environments, utilizing a continuous backdoor payload strictly as an experimental mechanism to observe how network geometry dictates parameter flow.

At its core, this research set out to answer a critical question: how does the physical layout of a network influence the survival and spread of a localized backdoor payload? By deploying a targeted threat across diverse architectural constraints, ranging from dense Fully Connected clusters to realistic, community-based Stochastic Block Models, this thesis provides empirical proof that network geometry acts as the fundamental variable governing adversarial propagation.

To enable a fair and mathematically sound evaluation, this research formulated and validated a topological boosting heuristic, $\alpha = d + 1$. By scaling the malicious parameter updates proportionally to the attacker's local topological degree, the adversary was able to perfectly neutralize the natural dilution caused by neighborhood aggregation. Rather than focusing on maximizing raw attack damage, this heuristic was utilized to maintain strict metric stability. The attacker consistently maintained a Clean Accuracy indistinguishable from the honest majority, successfully preserving the experimental baseline while exposing the underlying structural weaknesses of the graph.

The strategic placement of the adversary across these diverse structures yielded several critical insights into network resilience:

- **Density Accelerates Infection:** In a maximally dense Fully Connected topology, the absence of communication bottlenecks allows a continuous injection to easily overcome local dilution, driving a rapid and system-wide compromise.
- **Sparsity Delays Propagation:** Constrained topologies, such as the Ring, naturally throttle the infection. The physical necessity of sequential, multi-hop communication

slows the payload's progression, demonstrating that structural distance natively buys the network crucial time.

- **The Hub Vulnerability:** In a Star topology, centralized aggregation acts as an impenetrable shield against anomalous updates originating from the periphery. However, if the central Hub itself is compromised, the topology becomes exceptionally fragile, granting the attacker an unfiltered broadcast channel to instantly infect the entire network.

- **The Practical Optimum (SBM):** Evaluations using the Stochastic Block Model proved that community boundaries act as natural defenses, making it the ideal architecture for real-world deployment. It provides the rapid local convergence of a dense network, while its sparse gateway links act as natural topological firewalls. Even when the attacker controls the gateway node itself, the target community functions as a consensus sink, structurally washing out the malicious parameters and preventing global corruption.

Finally, by replicating these dynamics on the NEU-DET industrial dataset, this research confirmed that these structural vulnerabilities are inherent to the decentralized architecture, completely independent of the underlying complexity of the training data. Crucially, the industrial validation revealed that in highly complex environments, community boundaries shift from absolute firewalls to powerful temporal delays.

By trapping the payload for hundreds of rounds, the topology naturally buys the network the critical time needed to run diagnostics or deploy active intervention protocols.

7.2 Limitations of the Study

Like any controlled experiment, this research required abstracting certain real-world complexities to precisely isolate the variables under study. The primary limitations of this methodology are:

- **Scale of the Network:** To precisely observe the step-by-step mathematical dilution of the malicious payload, the experiments were conducted on an 8-node network. While this scale perfectly demonstrates the mechanics of peer-to-peer architectures, enterprise-level edge computing networks may consist of thousands of participating devices, which could alter the speed of global consensus.

- **Static Graph Architecture:** The evaluated network topologies were strictly static. In real-world environments, peer-to-peer networks are highly dynamic and characterized by churn, where nodes frequently disconnect, rejoin, or establish new lateral connections mid-training due to bandwidth or battery constraints.

- **Uniform Aggregation Assumption:** The formulation of the topological boosting heuristic fundamentally relies on the assumption of uniform neighborhood averaging, where honest nodes weight all incoming parameters equally. If a network were to employ dynamic weighting or trust-based aggregation algorithms, the multiplier would no longer perfectly cancel out the receiver's dilution. In such environments,

an attacker would have to dynamically estimate and adapt to the specific weights used by their neighbors.

- **IID Data Distribution:** The simulations utilized an Independent and Identically Distributed (IID) partitioning of both the MNIST and NEU-DET datasets. While this successfully controlled for data-driven variance, practical decentralized networks often suffer from highly heterogeneous (Non-IID) data distributions across different clients.

7.3 Future Lines of Research

Building directly upon the findings and limitations of this thesis, several exciting avenues for future research emerge in the domain of decentralized AI security:

- **Evaluating Non-IID Environments:** Future work must investigate how the $\alpha = d + 1$ topological boosting heuristic performs under severe data heterogeneity. It remains to be seen whether Non-IID data distributions act as an additional layer of natural dilution against backdoor payloads, or if the resulting chaotic loss landscape actually makes it easier for anomalous parameters to propagate.
- **Dynamic and Evolving Topologies:** Simulating the continuous backdoor attack on dynamic graphs is a crucial next step. Specifically, researchers should explore how community-based topological firewalls behave under network churn. For example, if a gateway node disconnects and the network dynamically reroutes its connections, it could inadvertently bridge an isolated attacker directly to the broader network.
- **Topology-Aware Defense Mechanisms:** Finally, while this thesis proves that structure alone can significantly delay continuous injections, future systems must develop active, topology-aware defenses. The design of Byzantine-robust aggregation algorithms tailored specifically for DFL, where nodes dynamically adjust their aggregation weights based on the trust or historical variance of their structural neighbors, remains a highly promising frontier for securing peer-to-peer AI infrastructure.

APPENDIX A

Detailed Individual Node Plots

This appendix provides the comprehensive grid visualizations of the individual node performances across all evaluated network topologies. For completeness and to facilitate direct comparison, the metrics for all 8 clients are presented simultaneously for each scenario. This includes the unedited, full 240-round progression for nodes that were previously highlighted in the main text, allowing for a transparent evaluation of variance, convergence speed, and payload distribution without distorting the narrative flow of the main experimental results.

A.1 Non-Adversarial Baseline Performances

The following figures illustrate the *Clean Accuracy* and *Distributed Loss* for each node operating under benign conditions over 240 communication rounds. These baselines establish the maximum potential utility and convergence stability inherent to each architectural constraint. The structural diagram of each topology is provided above its respective performance grid to serve as a node-placement reference.

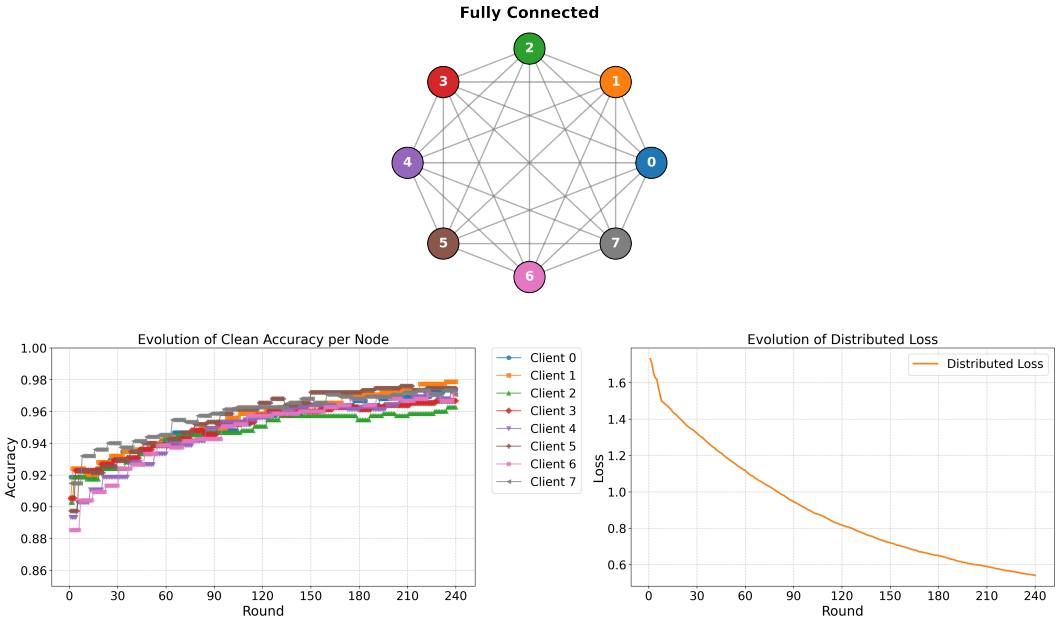


Figure A.1: Baseline performance of the highly dense Fully Connected topology. The node colors in the graphs match the structural diagrams above.

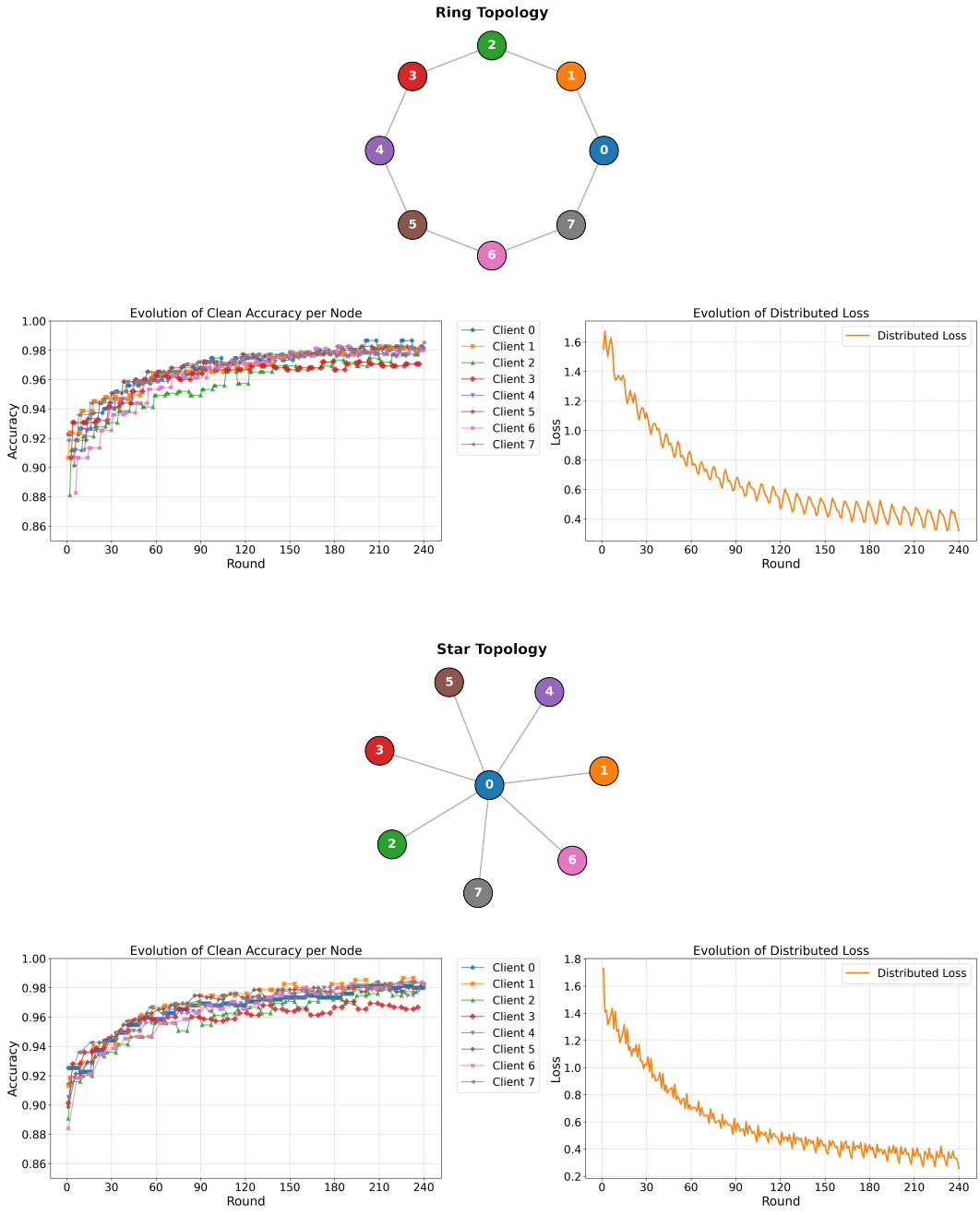


Figure A.2: Baseline performance comparison between the sparse Ring topology (top) and the centralized Star topology (bottom).

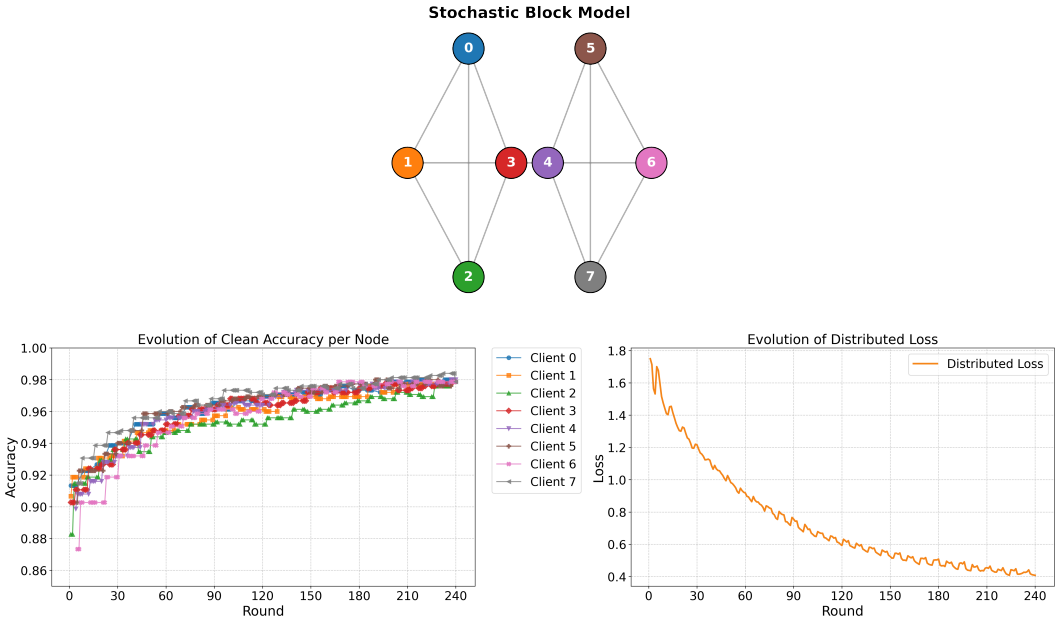


Figure A.3: Baseline performance for the Stochastic Block Model (SBM) topology, illustrating the natural convergence dynamics across two distinct communities.

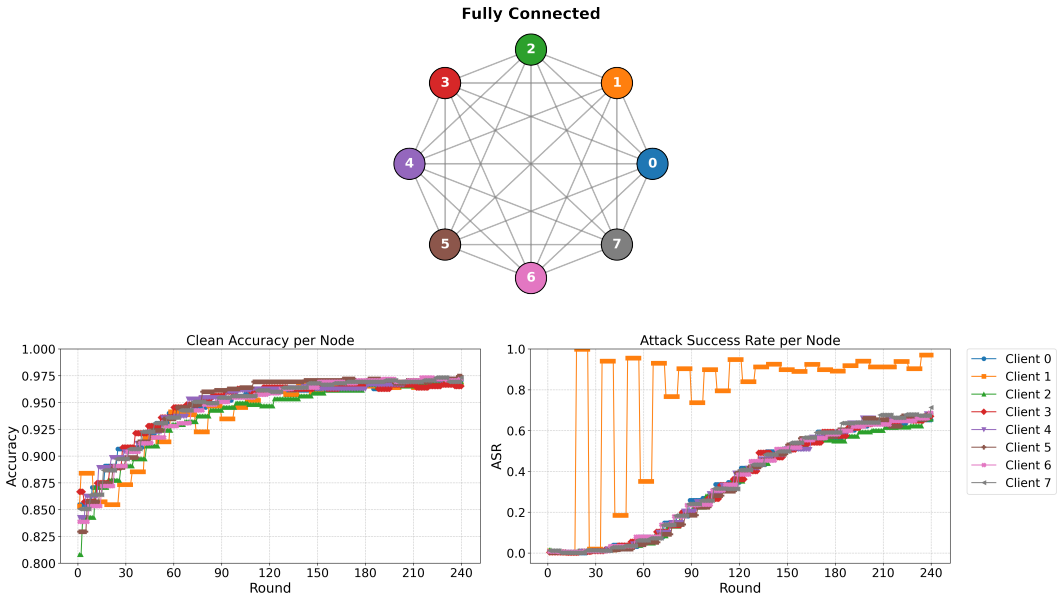


Figure A.4: Adversarial evaluation comparing a system-wide infection in the Fully Connected topology. The attacker is positioned at Client 1 (Orange).

A.2 Adversarial Evaluations

The following figure (Figure A.4) represents the complete 240-round performance metrics for all nodes under a continuous backdoor attack in the Fully Connected topology. The attacker is positioned at Client 1 (Orange). The top figure illustrates the network topology, with node colors matching the individual client lines in the subsequent plots. This comprehensive visualization allows for a detailed assessment of how the adversarial payload propagates through a maximally dense network, highlighting both the rapid infection dynamics and the uniformity of convergence among honest nodes.

APPENDIX B

Hardware and Software Environment

B.1 Hardware Setup

- **CPU:** 12th Gen Intel(R) Core(TM) i7-12650H
- **RAM:** 16 GB
- **GPU:** NVIDIA GeForce RTX 3050 Ti Laptop GPU

B.2 Software Frameworks

To ensure the full reproducibility of the experiments, the software environment was managed using virtual environments. The core frameworks and primary dependencies utilized in this research are detailed below:

- **Federated Learning Framework:** Flower (`flwr`) v1.7.0.
- **Deep Learning Framework:** PyTorch v2.5.1 (with CUDA 12.1 support).
- **Experiment Configuration:** Hydra-Core v1.3.2.

The exhaustive list of all dependencies, environment configuration files, installation instructions, and execution scripts is publicly available in the official project repository:

`https://github.com/IkerMardure/tfg-gossip-learning-backdoor`

In this repository, the `README.md` file provides the exact commands required to replicate the development environment and run the simulations presented in this thesis.

B.3 Simulation Runtime

The experimentation involves computationally intensive simulations of 240 communication rounds for both MNIST and NEU-DET scenarios. Runtime is dominated by repeated local training, peer-to-peer aggregation, and metric logging at each round, with additional overhead when adversarial evaluation is enabled. On the hardware described above, the full 240-round in the fully connected runs required approximately 24000 seconds for MNIST and 4000 seconds for NEU-DET. While the NEU-DET scenario utilized a significantly heavier architecture (ResNet-18), its total runtime was substantially lower due to the massive difference in dataset volume (1,800 total images compared to the 60,000 images processed per epoch in the MNIST baseline).

APPENDIX C

Declaration of use of generative AI

Tool/s used: Google Gemini Pro and GitHub Copilot

Purpose of use (tick as appropriate):

- ☐ Text generation
- ☒ Reformulation
- ☒ Translation / proofreading
- ☒ Structural suggestion
- ☐ Methodological support
- ☒ Help with coding/programming
- ☒ Other

Description of the use: I used generative AI primarily as a proofreading and formatting assistant to help polish the manuscript. Specifically, I used it to refine the academic tone, fix typographical errors, help optimize the code and optimize the LaTeX formatting. It was also helpful for getting general feedback on the flow of the chapters and the organization of the Table of Contents. All the core research, methodology, experimental design, data analysis, and final scientific conclusions are entirely my own original work.

Authorship commitment:
I confirm that all generated content has been personally reviewed, reworked and validated by me.

Bibliography

[1] Brendan McMahan, Ewan Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-efficient learning of deep networks from decentralized data. *Artificial intelligence and statistics*, pages 1273–1282, 2017. See pages 1, 5, 7, 8, and 14.

[2] István Hegedüs, Gábor Danner, and Márk Jelasity. Gossip learning as a decentralized alternative to federated learning. In *Distributed Applications and Interoperable Systems*, pages 74–90. Springer, 2019. See pages 1, 7, and 8.

[3] Paul Vanhaesebrouck, Aurélien Bellet, and Marc Tommasi. Decentralized collaborative learning of personalized models over networks. *arXiv preprint arXiv:1710.06175*, 2017. See pages 1, 7.

[4] Róbert Ormándi, István Hegedűs, and Márk Jelasity. Gossip learning with l2-regularized linear models. *Parallel Computing*, 39(12):769–785, 2013. See pages 1, 8.

[5] Qingqian Yang, Peishen Yan, Xiaoyu Wu, Jiaru Zhang, Tao Song, Yang Hua, Hao Wang, Liangliang Wang, and Haibing Guan. Stealthy backdoor attack in federated learning via adaptive layer-wise gradient alignment. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 29163–29172, 2025. See pages 1, 13.

[6] Georgios Syros, Gökberk Yar, Simona Boboila, Cristina Nita-Rotaru, and Alina Oprea. Backdoor attacks in peer-to-peer federated learning. *arXiv preprint arXiv:2301.09732*, 2024. See pages 1, 15.

[7] Eugene Bagdasaryan, Andreas Veit, Yiat Chia, Maxim Fedorov, and Deborah Estrin. How to backdoor federated learning. *International Conference on Artificial Intelligence and Statistics*, pages 2938–2948, 2020. See pages 2, 13, 14, and 19.

[8] Robert M French. Catastrophic forgetting in connectionist networks. *Trends in cognitive sciences*, 3(4):128–135, 1999. See pages 2, 13.

[9] Adam Piaseczny, Eric Ruzomberka, Rohit Parasnis, and Christopher G Brinton. Adversarial node placement in decentralized federated learning: Maximum spanning-centrality strategy and performance analysis. *IEEE Internet of Things Journal*, 2025. See pages 2, 14.

[10] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. See pages 2, 25.

[11] Ying Bao, Kebin Song, Jiyong Liu, Yan Wang, Yu Yan, and Hongkai Yu. A deep learning-based method for surface defect detection of hot-rolled steel strip. *The International Journal of Advanced Manufacturing Technology*, 102:399–405, 2019. See pages 2, 27.

[12] Lin Yuan, Zheng Wang, Lichao Sun, Philip S Yu, and Christopher G Brinton. Decentralized federated learning: A survey and perspective. *IEEE Internet of Things Journal*, 11(21):34617–34638, 2024. See pages 6, 7.

[13] Luigi Palmieri, Chiara Boldrini, Lorenzo Valerio, Andrea Passarella, and Marco Conti. Impact of network topology on the performance of decentralized federated learning. *arXiv preprint arXiv:2402.18606*, 2024. See pages 8, 15.

[14] Luca Melis, Congzheng Song, Emiliano De Cristofaro, and Vitaly Shmatikov. Exploiting unintended feature leakage in collaborative learning. In *2019 IEEE Symposium on Security and Privacy (SP)*, pages 691–706. IEEE, 2019. See page 11.

[15] Ehsan Hallaji, Roozbeh Razavi-Far, Mehrdad Saif, Boyu Wang, and Qiang Yang. Decentralized federated learning: A survey on security and privacy. *IEEE Transactions on Big Data*, 10(2):194–213, 2024. See pages 11, 15.

[16] Matt Fredrikson, Somesh Jha, and Thomas Ristenpart. Model inversion attacks that exploit confidence information and basic countermeasures. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, pages 1322–1333, 2015. See page 11.

[17] Yuheng Zhang, Ruoxi Jia, Hengrui Pei, Wenxiao Wang, Bo Li, and Dawn Song. The secret revealer: Generative model-inversion attacks against deep neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 253–261, 2020. See page 11.

[18] Giuseppe Ateniese, Luigi V Mancini, Angelo Spognardi, Antonio Villani, Domenico Vitali, and Giovanni Felici. Hacking smart machines with smarter ones: How to extract meaningful data from machine learning classifiers. In *Proceedings of the 26th Annual International Conference on Computer Science and Software Engineering*, pages 137–150, 2015. See page 11.

[19] Karan Ganju, Qi Wang, Wei Yang, Carl A Gunter, and Nikita Borisov. Property inference attacks on fully connected neural networks using permutation invariant representations. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pages 619–633, 2018. See page 11.

[20] Gyojin Han, Jaehyun Choi, Haehyun Lee, and Junghwan Kim. Reinforcement learning-based black-box model inversion attacks. *arXiv preprint arXiv:2304.04625*, 2023. See page 12.

[21] Nader Bouacida and Prasant Mohapatra. Vulnerabilities in federated learning. *IEEE Access*, 9:63229–63257, 2021. See page 12.

[22] Xiaofeng Xie, Chuanpu Hu, Hao Ren, and Jing Deng. A survey on vulnerability of federated learning: A learning algorithm perspective. *Neurocomputing*, 573:127225, 2024. See page 12.

[23] Guoliang Xia, Jiacheng Chen, Chunhua Yu, and Jianfeng Ma. Poisoning attacks in federated learning: A survey. *IEEE Access*, 11:10708–10722, 2023. See pages 12, 13, and 15.

[24] Ali Shafahi, W Ronny Huang, Mahyar Najibi, Octavian Suciu, Christoph Studer, Tudor Dumitras, and Tom Goldstein. Poison frogs! targeted clean-label poisoning attacks on neural networks. In *Advances in Neural Information Processing Systems*, volume 31, 2018. See page 12.

[25] Chen Zhu, W Ronny Huang, Hengduo Li, Gavin Taylor, Christoph Studer, and Tom Goldstein. Transferable clean-label poisoning attacks on deep neural nets. In *Proceedings of the 36th International Conference on Machine Learning*, pages 7614–7623. PMLR, 2019. See page 12.

[26] Clement Fung, Chris JM Yoon, and Ivan Beschastnikh. Mitigating sybil attacks in federated learning using sybil-based diversity. *IEEE Transactions on Information Forensics and Security*, 15:2423–2437, 2020. See page 14.

[27] Dong Yin, Yudong Chen, Kannan Ramchandran, and Peter Bartlett. Byzantine-robust distributed machine learning via distributed median and trimmed mean. *International Conference on Machine Learning (ICML)*, pages 5663–5672, 2018. See page 14.

[28] Rachid Guerraoui, El Mahdi El Mhamdi Guirguis, and Sébastien Rouault. The hidden vulnerability of distributed learning in byzantium. In *International Conference on Machine Learning (ICML)*, pages 1820–1829. PMLR, 2018. See page 14.

[29] Chulin Xie, Keli Huang, Pin-Yu Chen, and Bo Li. Dba: Distributed backdoor attacks against federated learning. In *International Conference on Learning Representations (ICLR)*, 2020. See pages 14, 18.

[30] Peva Blanchard, El Mahdi El Mghari, Rachid Guerraoui, and Julien Stainer. Machine learning with adversaries: Byzantine tolerant gradient descent. In *Advances in Neural Information Processing Systems*, volume 30, 2017. See page 15.

[31] Tianyu Gu, Brendan Dolan-Gavitt, and Siddharth Garg. Badnets: Identifying vulnerabilities in the machine learning model supply chain. *arXiv preprint arXiv:1708.06733*, 2017. See page 18.

[32] Daniel J Beutel, Taner Topal, Akhavan Akhavanniaz, Nicolas Marti, Yan He, Lorenzo Barkotto, and Nicholas D Lane. Flower: A friendly federated learning framework. *arXiv preprint arXiv:2007.14390*, 2020. See pages 23, 26.

[33] Aitor Belenguer, Jose A. Pascual, and Javier Navaridas. GLow - a novel, Flower-based simulated gossip learning strategy. *arXiv preprint arXiv:2501.10463*, 2025. See page 23.

[34] Paul W Holland, Kathryn Blackmond Laskey, and Samuel Leinhardt. Stochastic blockmodels: First steps. *Social networks*, 5(2):109–137, 1983. See page 25.

[35] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016. See page 27.

[36] Sheng Sun, Zengqi Zhang, Quyang Pan, Min Liu, Yuwei Wang, Tianliu He, Yali Chen, and Zhiyuan Wu. Staleness-controlled asynchronous federated learning: Accuracy and efficiency tradeoff. *IEEE Transactions on Mobile Computing*, 23(12):12621–12634, 2024. See page 30.